

Simulation

Performance Evaluation of Computer Systems and Networks

MSc in Computer Engineering

MSc in Artificial Intelligence and Data Engineering

Academic Year 2026/2027

Dr. Ing. Giovanni Nardini



UNIVERSITÀ DI PISA



Introduction to Simulation

Why should you use simulation?

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Most of the examples will be taken from computer networks. However, the concepts are quite general, and can be applied to simulating pretty much anything.

Performance evaluation of *what*?

“How does that new algorithm/protocol/method perform *better than* the previous ones?”

“Let’s evaluate the performance of that new algorithm/protocol/method!”



Performance evaluation of a **system**

“a collection of entities that act and interact together
towards the accomplishment of some logical goal”

- Identifying such *collection* does depend on the observer and their objective
- A system can be described through its *state*
 - The state changes over time → continuous vs discrete

A **system** is the subject of our performance evaluation, e.g. a computer, a network, a cyber-physical system

The collection of entities that describes the system depends on the objective of our evaluation -> might be a subset of the entities that compose the overall system in the real life

It depends on the observer: I might be interested in observing the performance of an application within the computer, or I might be interested in observing the average energy consumption of the computer’s circuitry. The set of entities considered in the performance evaluation may be different in the two cases.

State: set of variables – and their values – that describes the system at a particular time -> the state of the system changes over time

- State variables change continuously over time -> continuous system
- State variables change instantaneously at a given time -> discrete system

Performance evaluation of a system

It means **monitoring** the state of the system:



Evolution over time



Statistics



Limit values

It consists in **monitoring** some interesting quantities, how they **evolve over time**, observing their **statistics** (e.g. the mean value), or computing some **limit values** (e.g., the maximum value, etc.).

How to evaluate the performance of a system

Measurements

- Let the real system work (or set up a prototype of it)
- Measure the desired output

Analytical models

- Set up an equation-based model of the relevant aspect of the system
- Solve the equation-based model
- Get the results

Simulation models

- Write software that replicates the relevant aspects of the system
- Give it an input
- Run the simulation
- Measure the desired output

This can be done essentially in three ways (or any combination thereof):

a) Through measurement:

- you let the real thing work – no need to “validate” the system, but... do you have access to it? Can you manipulate it? Is that “safe”? Imagine you need to test a new process of a nuclear plant...
- you set up a prototype (e.g., buy hardware and write down the actual code), and have it run either “in the wild” or in a controlled environment, and take measurements.

b) analytically: you set up an **analytical model of the system**, i.e. **equations** that bind the quantities of interest. Remember that a system is represented by a state, which is a set of variables → building an analytical model means that you define the equations that define the relationships between these variables. In the end, you “just” solve these equations and end up with **exact** results.

- Problem: for **complex** systems, writing down these equations can be extremely difficult. For less-than-trivial systems, **solving** these equations is often impossible.
- You can **simplify** the models. You abstract away some of the details, and leave only the “important stuff”. The problem is that you never know in advance what is important enough, and you often end up underestimating the importance of the aspects you neglect.

c) Through simulation: you set up a **software replica** of your system; you give your software an input, and you get some output in return.

- If the system is complex, you can replicate only **a part of it** (or **some aspects of it**). If you want to evaluate a routing algorithm, you probably don’t need to simulate all the details of a router (and, more specifically, you don’t need to know any details on the switching fabric of the router, its memory access policy, etc.).

Which method is preferable?

It depends!

	Measurements	vs	Analytical models	vs	Simulation models
Level of detail					
Cost					

All techniques are useful, possibly at different stages of the design/evaluation process

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

6

Putting all together: you can build a simulation model of the system, you can validate it by comparing its results with an analytical model of it in a very simple basic scenario. After a complete simulation study, and you are almost ready to deploy your system, you can build a prototype of the real system.

What is a simulation?

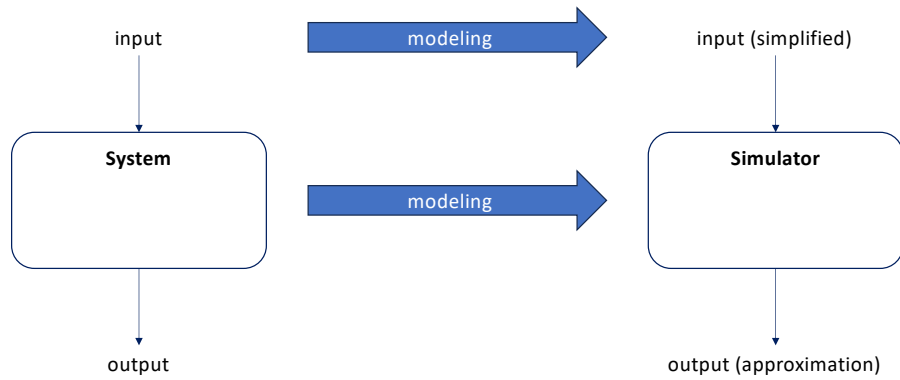
- Simulation means **mimicking a system on a computer**
- We need a software -> **simulation software** or **simulator**
- **Goals** of a simulator:
 - Understanding the functioning of the system
 - Predicting the behavior of the system
 - Supporting decisions
 - Training
 - Entertaining

These lectures are about simulation. Simulation is one of the most used performance evaluation techniques (besides *optimization*). In the field of computer engineering (and, more specifically, of computer networks), it is by far the most widely used one.

A simulator is a software. You are (or will be) computer engineers, and you know how to write software (even complex software). But writing a **simulation** software is not a piece of cake: it is not just coding (even though this plays a major role in the work), but you also need to understand a system, have good modeling skills (which you acquire with experience) and also statistics knowledge in order to manipulate the output of the simulation correctly.

It is also an active **research field**. There are journals and conferences dedicated to **simulation techniques**, and its scientific aspects.

From the system to the simulator



Careful **modeling** is key to good simulations

We need to obtain a **good** simulation model, as the reliability of the output will depend on how close the model is to reality

Our aim is to make the approximated output similar to the real output, or at least statistically equivalent.

Simulation's pros & cons

Advantages

- Almost everything can be simulated
- Allows you to evaluate alternative design choices **before going to production**
- Full control over **experimental settings**
- Allows you to observe the system at **reasonable timescales**:
 - Compress the evolution of systems with long time frames (e.g., years) to possibly a few seconds
 - Expand the evolution of those with short time frames (e.g., ns)

Pitfalls

- Stochastic quantities may be involved
 - A system's measurable characteristic is a **random variable**
 - Each output measure is a **sample** of the latter
- Often expensive and time consuming
- Requires considerable **expertise**
 - Not only coding, not by any means
- Relies on an **underlying system model**
 - Results cannot be more accurate than the system model
 - Easy to build false confidence

Almost everything can be simulated, provided that you are able to obtain a good model of it.

You can evaluate alternatives: what happens if the link between the endpoints of a communication is an Ethernet cable or a fiber optic cable.

Your “experimental settings”: when you do simulation, you simulate not only your system, but also **everything your system interacts with**. You can control your environment in a simulation setting, not quite so in a live measurement. A typical case is **scalability assessment**: you want to measure what happens if you multiply your input (e.g., a number of users) by a factor k . You can do this easily in a simulation, not so in real life. Another example is the interference in a wireless system: you can hardly control it in a real environment.

Timescales: you can simulate the evolution of the universe in a few seconds/minutes, but you can also observe how the instructions of a computer program are fetched and executed by a processor, which will typically take times in the order of nanoseconds

Stochastic quantities: assuming you are simulating the arrival of packets at a router, the **length** of these packets and their **interarrival** times will often be generated **randomly**, e.g., the simulator takes as input **data sampled from probability distributions**. This means that the **node throughput** is itself a **random variable**. Each measure that is obtained from the simulator is a **sample** of an **output random variable**, and should be treated as such.

Writing down a simulator is no piece of cake. Writing down the simulator of the LTE technology took 2 highly specialized PhD students, and resulted in 40k+ lines of code. This was obtained **relying on a pre-existing** (general-purpose) **simulation framework**, i.e. customizing something that was already available.

Furthermore, **simulation analysis** (not *writing down the code*, but **using that code**) requires **a lot of time**, and (mostly) **a lot of experience**. You need to know: a) the system to be analyzed, b) probability theory, c) statistics, d) computer science, which makes simulation a typical **interdisciplinary field**. Never think that simulation analysis is all about **writing code**. This is just the beginning, and it is the **easiest part** for a computer engineer.

It is difficult to realize that a simulator is a replica of **a model of a system**, and not **of the system itself**. A model is a **simplified** thing, where you neglect some aspects, and make abstractions. If, when doing the **modeling part**, you take some **wrong modeling steps**, the fact that you make everything else state-of-the-art is completely **pointless**. The results that you obtain will have nothing to do with reality.

A simulator can normally be employed to produce a **huge amount of information**, e.g. estimates of many interesting quantities. It is easy to build **false confidence** on the correctness of the data when you are faced with a lot of data. You can only be certain when **the same results** are obtained in the **real system**.

Different types of simulation models

Simulation means different things to different people

1. Static vs Dynamic
2. Continuous vs Discrete
3. Deterministic vs Stochastic

1 – Static vs dynamic simulators

“Will the bridge support *this* weight distribution?”



“What are the characteristics of the traffic flow?”

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

11

Static:

- Model of a system at a given time.
- The output is obtained as $y = f(x)$, where f is the model and x is the input. Time is not a factor
- Can be used to solve differential equation systems, or to understand input/output relationships in complex systems.

Dynamic:

- we are interested in observing how the model changes over time
- replicates something that is supposed to **happen as time progresses**.

2 – Continuous vs discrete simulators

“How does the speed of the car vary over time?”



“How many customers will be waiting in line?”



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

12

You can model a continuous system as discrete - and vice versa - depending on the objective of your evaluation... but be careful:

- Modeling the speed of a car as discrete -> may lead to inaccurate results
- Modeling the number of people in the line as continuous -> may lead to having 3.4 people in line

3 – Deterministic vs stochastic

“How much will my mortgage cost me in total?”

Fixed interest rate



Variable interest rate

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

13

Deterministic: no random components are present in the model. Given a set of input, the model will produce (possibly, after long computations) a unique set of outputs

Stochastic: random components are present. Given an input, the model produces a set of random outputs. Thus, the model only produces an estimate of the output of the real system.

Discrete Event Simulators (DES)

- Dynamic, discrete, stochastic
- DES nicely fit the context of computer systems and networks
 - Intrinsically discrete from our point of view
 - CPUs execute instructions
 - Data is broken into packets
 - Might be considered continuous from an electrical/electronic point of view

Simulation of a computer system/network:

- **Dynamic:** evolve over time
- **Discrete:** requests/packets arrive at discrete time intervals
- **Stochastic:** the inter-arrival of requests/packets is a random variable

How to build a simulator

aka “How to build a GOOD simulator”

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Basic data structures

- A system can be described through its state → **state variables**
 - A collection of variables that describe the system at a particular time
- The state of a system changes over time → **simulation clock**
 - A variable containing the current value of (simulated) time

When building a simulator, you need to represent the state of your system, that is you need to foresee a set of **variables storing values that represent the current condition of the system**

Since we are considering discrete-event simulators, which are dynamic according to our earlier definition, one of these variables must contain the current time, aka **simulation clock**

Real time vs. simulated time

- In a dynamic simulation, the system evolves in **simulated time**
 - During one second of simulated time, the simulator does something
 - e.g., update the state of the model
 - The computer's processor needs to execute the corresponding instructions
 - Such instructions take some **real time** (wall-clock time) to execute
 - How much?
- In general, **no constant relationship** between the flow of simulated time and the flow of real time
 - More state's updates -> longer real time
 - Fewer state's updates -> shorter real time

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

17

We are talking about **dynamic simulators**, i.e. those where **time** plays a role. A dynamic simulator is a piece of code that replicates something that is supposed to **happen as time progresses**.

Simulated time: the time as measured **within** the simulation (the time at which something happens in the simulator). This is not the **real time** (otherwise you talk about **emulation**, not **simulation**). In order to simulate one second of simulated time, you may need either more or less than one second of real time, depending on what happens in the simulated system.

In general, the relationship between the two depends on what the system is doing during that span of simulated time:

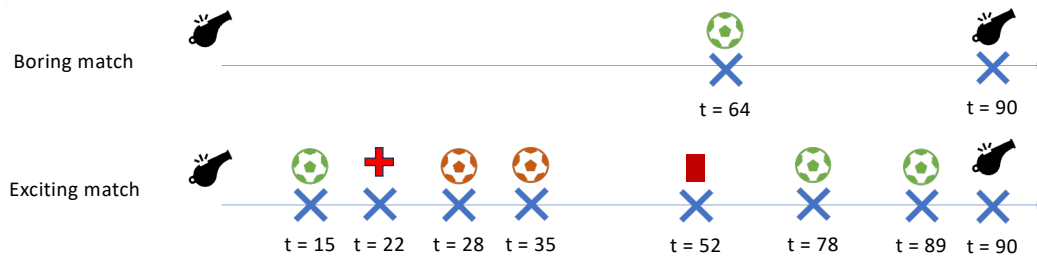
-Many events -> a larger span of real time

-Few events -> a smaller span of real time per second of simulated time.

Example: to simulate 200s of activity of a 4G cellular network, our simulator takes from 10s to some hours, depending on how many users are in the cell.

Real time vs. simulated time: an analogy

Consider the highlights of a football match, which show only relevant events



- **Match clock = simulated time** → always 90 minutes
- **Duration of the highlights = real time** → the time you spend watching
 - It depends on the number of relevant events in the match and on the length of each replay

Events

- Everything that **changes the state** of a simulated system
 - e.g., arrival of a new request to a server, departure of a packet from a network node
- The simulation time advances based on the sequence of events

Next-event time advance



Fixed-increment time advance



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

19

How do you make time evolve in a simulator? Internally, the simulator keeps a **simulation clock** variable, storing the current simulated time.

Such simulation clock can be advanced in several ways:

Next-event time advance (or event-based): simulated time advances when an event is processed. The system processes an event at time t_1 (e.g., the arrival of a packet at a node). Processing this event implies **generating** another event (e.g., its departure from the node) at time t_2 . The system's clock (i.e., simulated time) advances based on the event sequence. Therefore, it is clear that the **real-time duration** of one second of simulated time varies depending on how many events are in that second (and what kind of processing they require).

Fixed-increment time advance (or quantum-based): the system clock advances by a fixed quantum (of simulated time). At time $2D$, you process all the events that occur between D and $2D$. This implies two problems and one advantage:

- If the **granularity of the events is $\gg D$** , you waste a lot of (real) time just to advance the clock when nothing happens. With event-based simulators this would not happen
- If, on the other hand, the granularity is **$\ll D$** , events that are very distant are processed at the same time, hence they are treated as **simultaneous**.
- However, in this case, you can use a **discrete variable** as a temporal quantum (i.e., an integer instead of a float, which is useful for implementation purposes).

Fixed-increment time advance makes sense if the system is **intrinsically time-discrete** (e.g., ATM, where time is divided into slots). Otherwise it gives you more problems than benefits.

Moreover, fixed-increment time advance can be recreated using next-event time advance mechanism, by scheduling "clock events" at every DT .

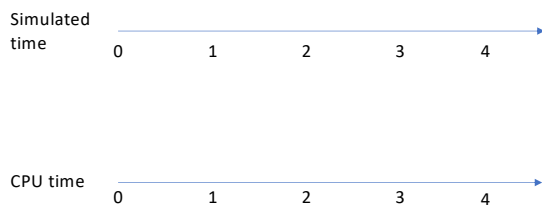
For these reasons, from now on we will assume that our discrete-event simulators use the **next-event time advance mechanism**.

I mentioned that "you waste a lot of time": in simulation, **optimizing the system performance** is extremely important (serious simulation studies may last for weeks). The running time of a simulator is something to care about right from the start.

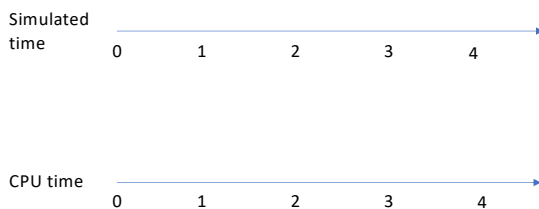
Handling events takes time

Assume that handling one event takes 1 second of real (wall-clock) time:

- $t_1 = 0s \rightarrow$ event 1
- $t_2 = 3s \rightarrow$ event 2



- $t_1 = 0s \rightarrow$ event 1
- $t_2 = 0.5s \rightarrow$ event 2
- $t_3 = 1.2s \rightarrow$ event 3
- $t_4 = 2s \rightarrow$ event 4



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

20

Simulated time advances in a different way with respect to real (wall-clock) time.

Since **handling an event means executing some instructions to update the state of the model**, the time required to process an event is non-zero.

In these examples, we observe two different cases:

- a) simulated time is faster than real (wall-clock) time
- b) simulated time is slower than real (wall-clock) time

We implicitly assumed that each event takes the same time to be handled. This may not be true, as different events will need to execute a **handler function that modifies** the state of the system: such handler can contain more or less instructions, depending on the type of the events and the detail of your model.

Modeling something that takes time



Real system
0 1 2 3 4

Simulated time
0 1 2 3 4

CPU time
0 1 2 3 4

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

21

In a dynamic simulator, each event updates the state of the model. Such update occurs **instantaneously**, from the point of view of the simulated time.

What if we want **to simulate a phenomenon that takes time**? For example, sending a packet over a wire, writing a file to disk, serving a request at a server

Example:

Assume that writing a file on a disk in a real system takes 1 second. In our simulator, we model this as two events:

- "start write" event at $t=0s$,
- "end write" event at $t=1s$

Assuming that processing one event takes one second of CPU time, then our CPU time will be $t=2s$ at the end of the simulation. If another event is generated at $t=0.5s$, then our CPU time will be $t=3s$ at the end of the simulation.

A write-on-disk event will modify the state of the system by modifying the amount of occupied memory: when do I update this value? In the «start write» or «end write» event?

Event queue

- An event is represented by a data structure including a **firing time** field
- Future (outstanding) events must be kept in a list of events
- **Event queue**
 - Events are sorted by firing time
- Supported operations:
 - Extraction of the nearest future event
 - Ordered insertion of a future event
 - Deletion of a future event

Event
firing time
event type
other data

The simulation software will handle the event in sequence, so where does the CPU find those events? Events are stored in an event queue.

Firing time is the simulated time at which the event is intended to occur, i.e., when the event is fetched the simulation clock is set equal to that firing time.

The event type specifies how the event must be handled, i.e. defines the handler function to be executed when the event is processed.

Events in a simulator are handled in an **event queue**, which has to be implemented in the **most efficient possible way** (it is the critical point for the performance of a simulator).

Event handling is a critical issue for system performance. Depending on how efficiently the queuing/dequeuing of events is performed, the (real) time that it takes for a simulation to run can easily change by **orders of magnitude**.

Statistical counters

- The purpose of simulating a system is evaluating its performance
- This means observing some statistics of interest
- The simulator should maintain some **statistical counters**
 - Collection of variables that store values useful for assessing the performance of the system under evaluation

Can be scalar values (e.g., min/max/mean values, standard deviations, etc.), time series (e.g., evolution over time of a given metric), histograms.

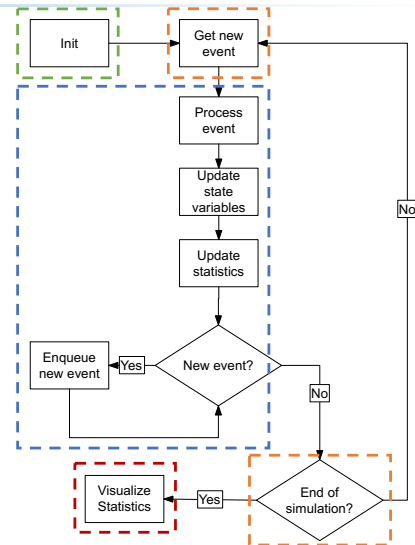
Components of a simulator

- Data structures
 - State variables
 - Simulation clock
 - Event queue
 - Statistical counters
- Functions
 - **Initialization**
 - Assigns the value of state variables at time $t = 0$
 - **Event scheduler**
 - Determines the next event to be processed, by extracting it from the event queue
 - **Event handlers**
 - Updates the state & statistical variables
 - Possibly inserts new events in the event queue
 - **Statistics computation & visualization**
 - Computes final values of the statistics, and produces them as output of the simulator

Initialization includes all the functions to initialize state variables, reset the statistics, parse command line options and/or configuration files (if any)

Components of a simulator

- Data structures
 - State variables
 - Simulation clock
 - Event queue
 - Statistical counters
- Functions
 - Initialization
 - Event scheduler
 - Event handlers
 - Statistics computation & visualization



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

25

The activity of a simulator consists of a continuous cycle where:

-You extract the nearest future event from the queue, thus advancing the simulated time (**clocking function**). This element is critical as one has to prevent the creation of an event in the past (i.e. maintaining a causal system)

-You process that event using an **event handler**. In doing this:

- You update the state of the simulator,
- You update the statistics that you want to observe at the end of the simulation
- Possibly, you generate one or more new events, and insert them into the event queue

-At the end of the simulation, you visualize the statistics of interest.

We will learn how to implement simple versions of these components in the first labs, but when doing your project you can focus just on implementing the model of your system, because most of these components are already provided by OMNeT++ (which does that in a very efficient manner)

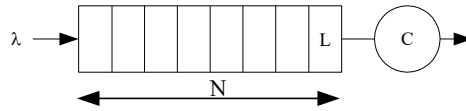
You will typically need to implement your own simulation components only in special cases, e.g. when your problem is very specific and you have strict requirements on efficiency of the event queue.

General purpose vs ad-hoc simulators

General purpose simulators provide generic implementation for the event queue, the event scheduler, statistics recording etc. (i.e., a common simulation engine). This is good for most of the systems you may want to model. In this case, you just focus on the model of the system you want to evaluate.

In very specific cases, you may want to exploit the characteristics of the system to optimize the performance of the simulator, e.g. you know that most of the events will be inserted at the front of the event queue -> you implement your own event queue, optimized for front insertions. Most likely, such simulator will not be useful to simulate another system.

A simple example – problem definition



- Single queue system, e.g., a network interface
- We want to know the **average delay** of packets in the queue
- We assume:
 - Packet length is constant and equal to L
 - Link bandwidth is C
 - The queue contains at most N packets
 - The processing time of the scheduler is null
 - Inter-arrival time of packets is a random variable $X \sim \text{Exp}(\lambda) \rightarrow E[X] = 1/\lambda$
- We either **solve the system analytically** or **build up a simulator of the system**

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

27

We consider a network interface as a simple example, although the same modeling can apply to a server, a post office, and so on.

Inter-arrival time is a random variable following the exponential distribution with rate λ , resulting in a mean of $1/\lambda$

This means that **also the average delay is a random variable**: when I run the system twice I get a different sample for the average delay (because packets arrive at different times)

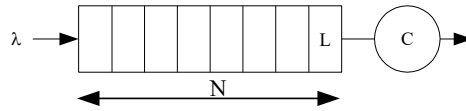
With queueing theory, we can build an **M/D/1/N** model:

- M: arrivals follow the Poisson distribution (M stands for Markov process - memoryless)
- D: service time is deterministic
- 1: single queue
- N: system with finite capacity of at most N elements
-

What are the state variables we should maintain here?

And which statistical counters do we need to compute the average delay?

A simple example – initialization



- Simulation clock
 - $t = 0$
- State variables:
 - Packets in the queue = 0
 - Busy status = false
- Statistical counters:
 - Packets served = 0
 - Sum of delay = 0
 - (Zero packets dropped)
- Configuration parameters to be read (from command line or config file):
 - Values of N, L, C, λ

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

28

If you want to compute the mean delay, you should keep **two counters**: one for the number of **transmitted packets**, one for the **sum of the delays**. The latter are to be incremented on every packet departure. At the end we will output the ratio of the second to the first.

Of course you can also maintain other statistics (i.e., the number of dropped packets, etc.) for future use.

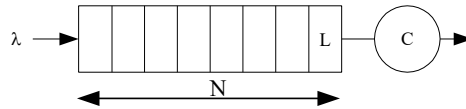
In a simulator that has some more functionalities than the toy example we are using as a reference, **configuration parameters** may be in the **tens or hundreds**. In this case, it is extremely impractical to pass them through the command line.

For this reason, you normally define **scenario files**, where you store, using a **textual syntax**, the values of the parameters that are used in a simulation run.

The syntax **should** be textual, because scenarios are indeed written by a user, and the user should be able to read them again after some months and remember what she did without wasting too much time.

In any case, you also need some code to **parse input configuration parameters**, either from the command line or from a configuration file. Parameters are **never hardcoded** within the simulator (otherwise you would be forced to recompile it every time a scenario changes).

A simple example – events



- We should ask ourselves: what are the (relevant) events in our system?
1. A new packet arrives in queue
 2. A packet goes under service (starts transmission)
 3. A packet leaves the system (ends transmission)
 4. (implicit) Simulation ends

Why should **these** be events?

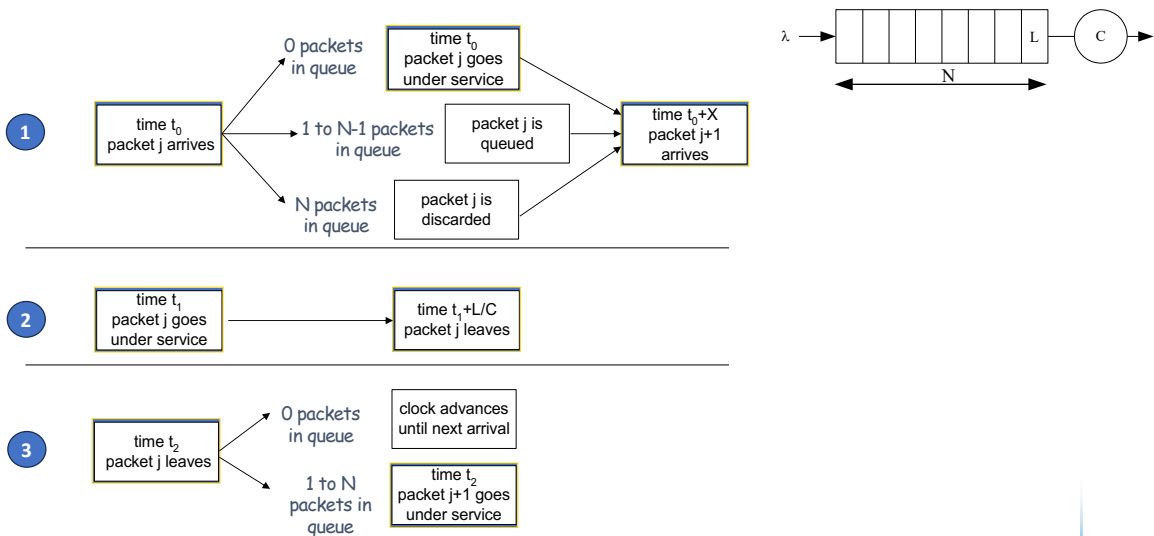
Because they **change the state of the system**.

Example of non-relevant event: incoming packet at the network interface from the network

The “end of simulation” is normally modeled as an event, which is inserted in the event queue at the beginning of the simulation (this is part of the init function). This is an event because you may want to handle it in a special way (e.g., to compute “a posteriori” statistics, etc.).

Since every event has its own handler, it is customary to make all end-of-simulation processing part of the handler for the “simulation end” event. In our case, that handler should compute the ratio between the sum total of delays and the number of transmitted packets and print it (plus some other tasks, possibly).

A simple example – event handlers



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

30

Events are **circled in yellow** in this picture. Handling these events may generate more events.

Observation: given that the length of packets is constant, a type-2 event could also be eliminated. More so if you consider that a type-2 event always generates a new type-3 event at a constant temporal distance, whereas a type-3 event also generates a new type-2 event at a constant (null) temporal distance. Why keep these separate, then?

because, in the future, you may want to **modify your simulator** so as to take into account variable-sized packets, and you will want to do that without re-writing it anew.

What data structure should I use to store the events? For a system like this, the most efficient choice would probably be to devise an ad-hoc data structure. For instance, if $L/C \ll E[X]$, it is very likely that

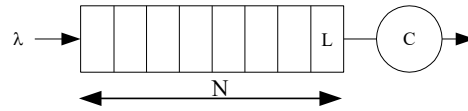
- Type-1 events are **tail** insertions
- Type-2/3 events are **front** insertions.

This knowledge could be exploited to make the system more efficient.

Take care not to **optimize too much**, because the more you optimize, the less flexible your simulator becomes. You have to strike a **trade-off** between flexibility (reconfigurability) and performance.

There are, however, safe choices which are near-optimal in most cases.

A simple example – statistics computation



- At the end of the simulation, we have obtained:
 - Number of packets transmitted
 - Sum of delay
- Thus, we can easily compute the **average delay** of packets
- Can we measure the **variance**?

If we would like to obtain also the variance, the recorded information is not enough -> we would need to keep all the delay values (e.g., in a vector)

Implementing the event queue

- A discrete event simulation spends a lot of time handling the event queue
- Efficient implementation is key to fast and scalable simulations
- Commonly employed data structures
 - **Min-heap tree**
 - **Calendar queue**

Steps involved in a simulation:

- 1- Get the next event in the queue
- 2- Execute the event
- 3- Add new event
- 4- Go back to step 1

If you profile your code using a profiler (e.g., Valgrind), you quickly discover that a significant portion of real time is spent by handling the event queue.

Need to optimize common operations in the queue: **extraction** of the next event, **insertion** of a new event, deletion of an event

Min-heap tree: what is it?

- Binary tree with the following characteristics:
 - The value of a parent node is less than or equal to the value of each of its children
 - Each level is filled from left to right, level $n+1$ cannot be populated unless level n is completely full
 - Thus, **depth** $\sim \log_2(\text{\#nodes})$

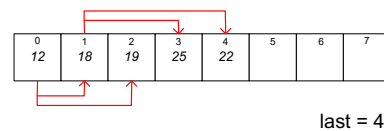
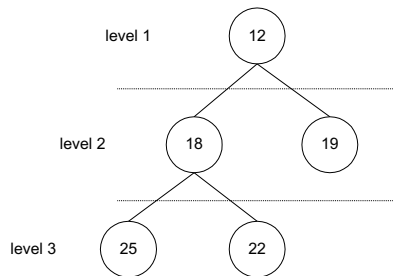
The basic observation here is that you do not need to keep **all the events sorted**: you just need to be able to find the **next in time** quickly. For this, you can use a min heap.

In this case, we use the **event firing time** as a sorting key -> the root of the tree represents the next event to be executed.

Note that, if you preserve the property of **quasi-completeness** when making insertions/extractions, the tree depth is **logarithmic** with the number of nodes (events). The complexity of the functions operating on this min-heap tree goes with the depth of the tree, hence it will be logarithmic as well.

Min-heap tree: implementation

Can be implemented with an **array** and a **counter**



For each node j

- Children at position $2j+1, 2j+2$
- Parent at position $\text{floor}((j-1)/2)$

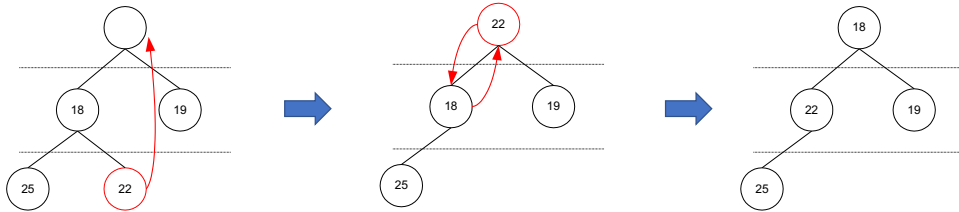
The counter is needed to know how many positions are occupied in the array.

A min heap of a **known maximum size** can be implemented very efficiently:

- Compact memory (just need to store the values, i.e. no pointers)
- Can go from one node to its parent/child in constant time, using the formulas

Min-heap tree: extracting the next event

- Extract the **root node**
- «Reheapification» → recover the min-heap properties
 1. Select the «last» node as the new root
 2. Swap with the minimum between the two children of the root, if root is greater
 3. Recursively delve into the tree and do the same with each sub-tree



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

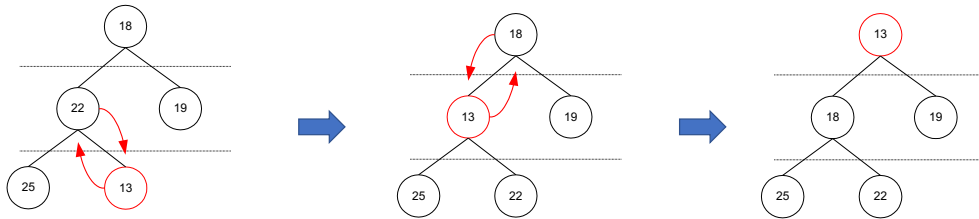
35

The extraction of the nearest future event is **the extraction of the root node** → $O(1)$

After the root has been extracted, **reheapification** occurs, i.e. the min-heap property is obtained again → $O(\log N)$

Min-heap tree: insertion of a new event

- Insert the node at the **first free place** (last level, left to right)
- Reheapification
 1. Swap – if needed – the newly inserted node and its parent
 2. Repeat the same procedure until the root node



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

36

The **maximum** cost of an insertion/extraction in a minheap is $O(\log N)$, since it is at most equal to the number of levels among which you need to swap. This is because the tree is quasi-complete.

What about **deletion of an event from the heap**?

- Once you locate it, cancel it and put the **last event on the bottom level** in its stead (thus preserving quasi-completeness)
- Reheapify (either upward or downward), which is still $O(\log N)$

So, is deletion $O(\log N)$ as well?

The answer is no, since you need to **locate** the element first, and this takes $O(N)$. This is not a **search** tree, and the min-heap property is not enough to locate an element in log time.

Bottom line: if you need frequent event deletion, don't use a min heap.

Calendar queue: what is it?

- Array of M **buckets**
 - A bucket is a *sorted list* of events
- All buckets have the same *width* δ
 - Bucket as a month of a calendar, events as the appointments in that month
- An event with firing time t is placed in bucket $i = \left\lfloor \frac{t}{\delta} \right\rfloor \bmod M$
- Events within the same bucket are sorted in time

- Worst-case complexity?
- Average-case complexity?

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

37

If the events are distributed more or less uniformly, the probability that you find an “empty” calendar day (bucket) is quite small.

Moreover, if the calendar year is «long», then the number of collisions of events of different years on the same day is low.

-> the number of operations to perform in order to locate the next event is low **on average**

In a worst case, it may happen that:

- All events are in the same bucket – $O(N)$ cost in a linear queue, $O(\log N)$ if min-heap is used
- You need to cycle through $O(M)$ empty buckets

So the calendar queue has a **worst-case** cost of $O(N+M)$ – or $O(\log N+M)$, although the **average** number of operations required to insert/extract events is quite small.

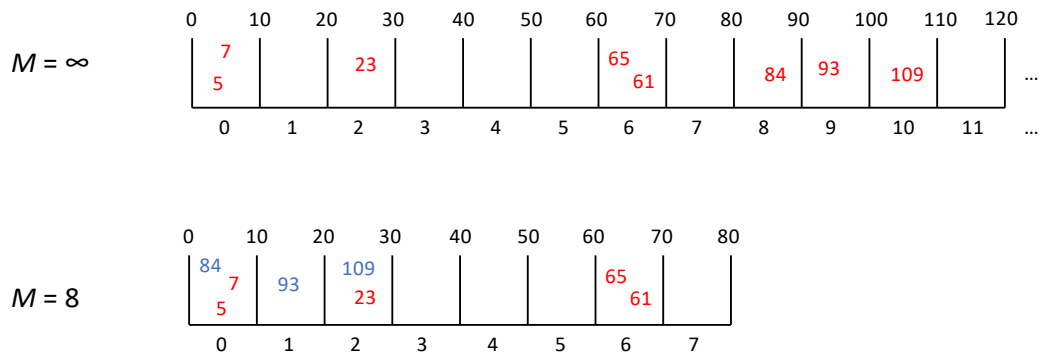
Which of the two should I pay attention to, the worst-case or the average cost?

It depends on what you need to do

- a) worst-case cost <-> you have to guarantee a **maximum execution time** (e.g., a real-time system)
- b) Average cost <-> you have to minimize the overall running time of a system (e.g., a simulator)

Calendar queue: example

- $\delta = 10$



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

38

Events in bucket 0 have to be ordered (e.g., using a min heap).

Events queued on each bucket have to be ordered w.r.t. their firing time.

If:

- Events are **evenly distributed** in the buckets
- There aren't too many **consecutive empty buckets**

Then it is quite easy to locate the next event to be scheduled, and it is not computationally expensive to keep each bucket's queue sorted (the cost of sorting grows with the number of events to be sorted).

Note that there is no ambiguity related to the fact that the same calendar page (bucket) contains events related to different years (e.g., 23 and 109 in bucket 2). If bucket 2 is the current bucket, and 23 is the next event to be scheduled, it is perfectly obvious that 109 is an event for the next year, and should only be considered at the subsequent round, after the event scheduler has cycled all the way back to bucket 2. You just need to maintain a **year counter**

At year j :

If (firing time of top event in current bucket) $\geq j * M * \delta$

Then

increase bucket // not in the current year

Else

pick next event from current bucket

In the example, 84 is greater than 79, which is the maximum day in year 1

Calendar queue: observations

- On average, you need to sort just a **small subset** of events (within each bucket)
- A lot of operations to be done if there are a lot of consecutive empty buckets
- Inefficient if δ and M are chosen poorly
- Can be dynamically adjusted
 - Number of events is twice the number of buckets $\rightarrow M = M * 2$
 - Number of events is half the number of buckets $\rightarrow M = M / 2$
- Relocation problems?

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

39

Delta and M can be chosen

-Statically

-Dynamically

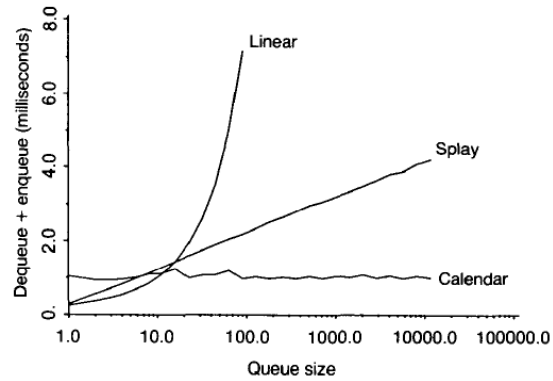
If they are **static**, then the efficiency of a CQ is determined by their value as related to the granularity of events (there are optimal choices if you know how your events are interleaved in time). More specifically, if you know the **event hold time**, i.e. the RV that measures the time between the *scheduling* and the *firing* of an event, then you can compute optimal values for the two parameters. Just Google “optimizing calendar queues” to see some of the available literature on the subject.

If they are **dynamic**, the queue can be dynamically adjusted. For instance, the # of buckets can be adapted to the # of events (with some hysteresis, lest it oscillates too often). There are two strategies to do this:

-**freeze**: when you want to use a different M , you stop the simulation, build the new CQ, move all the events from the **old queue** to the **new queue**, and then you resume (cons: you waste time managing your data structure while simulated time is, in fact, frozen).

-**Gradual**: you build the new queue, and start populating it with the new events. While you do this, you keep extracting events from the old CQ, until it is empty, and then discard it. The problem is that you still need to check whether the **next event** is in the **old queue or in the new queue** (and you have to move the “end of simulation” event in any case)

Calendar queue: efficiency



R. Brown, "Calendar Queues: A Fast $O(1)$ Priority Queue Implementation for the Simulation Event Set Problem", 1988

Linear: min-heap implemented with linked list

For larger queues, calendar queue performs better

Giving inputs to the simulator

- **Trace-driven** simulation
 - Input is taken from a trace obtained by measuring the real system (or something quite close to it)
- **Self-driven** simulation
 - Input is *artificially generated*, by generating random inputs from a distribution
 - Theoretical distributions
 - Empirical distributions
- Mixtures of both approaches are possible, in case of multiple inputs:
 - MPEG video packets (obtained from a real video file) transmitted to nodes that are moving according to a theoretical distribution
 - Exponentially-distributed packets transmitted to nodes that are moving according to predetermined trajectories (based on real system observations)

There are two basic approaches to generate input to a simulator

- Trace-driven** approach: you obtain a trace of the input of the system over time, and you feed it to the simulator. In our case, this means to obtain a file listing (time, bytes) for every packet arrival. Typical case in networking: **compressed** videos, since they are difficult to model.
- Self-driven** approach: this is what we have assumed in our example. It consists in generating input through **probability distributions**. Our approach was to use a **theoretical** one, i.e. the exponential. This is because, of course, we have **measured some data and fitted it to an exponential model**, or we are knowledgeable enough to state that the interarrival times can be expected to be exponential.

In our toy example (queue+server) input is **artificially generated** (synthetic input, self-driven simulation). In order to generate that input I need **random number generators** (recall that interarrival times have to be exponentially distributed).

There are cases when **you can't fit your data** to a known distribution (either because there isn't one, or you can't find it). In these cases, you may resort to **extracting a distribution from the PMF of your data**. This is called using **empirical distributions**. This should be done as a last resort.

Trace-driven simulation

- Example: MPEG-compressed video
 - List of tuples with format: `<time, video frame size>`
 - For each tuple generate an event at the given time that triggers the transmission of a packet of the given size
- **Pros**
 - Real-life case study -> scalability
 - Good for validation purposes
 - Might be the only option
- **Cons**
 - Overhead, as you need to obtain a trace (time consuming)
 - Storage, as traces are usually stored on disk and might be huge (also time consuming to read from disk)
 - Lack of generality, as traces may be very specific (e.g., different videos have different traces)
 - Inflexible, as traces cannot be tuned (e.g., to artificially increase the sending rate/traffic load)

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

42

Example of a **trace-driven simulation**: if I want to test the performance of a scheduling algorithm when transmitting **telnet** packets, I can sit at my desk, open a telnet session somewhere, intercept the packets that are transmitted by my system and generate an **input trace** in a table form such as:

Time	length
0.01s	150 bytes
....	

Afterwards, I can **use that trace file** to run my simulation. Whenever I need to generate a new “packet arrival” event, I can read the arrival time and packet length from the next line of the tracefile.

Real traces **normally give more reliable information** on the working conditions of your system. However, you need **experience and common sense** in order to be able to choose wisely.

A simulation study is often aimed at inferring **general properties** of a system, which are true under widely general assumptions. A trace-driven simulation says that your system performs (say) better than another **under those specific settings**, which may be hardly general. Especially with videos, an action movie is very different from a sports event or a talk show program. Nevertheless, it may be the only way to do anything meaningful with some kinds of traffic (notably: compressed video), since synthetic generators are not up to the job.

Traces cannot be tuned: if you have a trace of a low-quality video, and you need a trace of the same video at a higher resolution, you **cannot scale your trace up**. You will need to encode it anew. The same occurs if you have a trace of n customers, and you need a trace for $2n$ customers: you cannot just halve the interarrival times, because it will not work.

Moreover, traces may include too few data to produce a long-enough simulation.

Self-driven simulation: theoretical distributions

- Choose a probability distribution that closely mirrors the phenomenon of interest
 - e.g., packet inter-arrival time → exponential
- How to select the right one?
 - Gather data from the real system, perform statistical goodness-of-fit tests, find the right distribution and its parameters
 - Read existing papers, if any
 - Use common sense and knowledge of the problem
- **Pros**
 - Compactness, known statistical properties, tunability
- **Cons**
 - Identification of the right distribution may be complex

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

43

The opposite w.r.t. the previous case:

- I can **modify the distribution** by varying its parameters (e.g., ask myself what happens if packets arrive 10% faster). I can't do this using a tracefile: if I want to change the frame-rate of an MPEG movie (or its definition), I have to **re-encode** the entire movie, since scaling is not linear (not by a long shot)
- Artificial distributions require **little information** to be stored.

How do you choose the right one? Either you do the work yourself, or you **rely on the literature**: if the problem is real, it is highly likely that someone else has already dealt with it, and you can re-use her results (with a grain of salt, as usual).

The advantages of a theoretical distribution are several:

- Compactness (you can generate it with very few lines of code usually – more on this later)
- Known statistical properties (it helps you to validate your simulator)
- tunability

Self-driven simulation: empirical distributions

- Derive a(n empirical) probability mass function (EPMF) from available samples
 - Problem: selection of ranges and bins (more details on this later)
- Beware of censored data
 - When deriving an EPMF, you might have overflowing samples (more details on this later)
 - You may want to take into account the probability to generate larger values than the range defined for the EPMF → add an exponential tail to the right
- **Pros**
 - Sometimes it may be the only viable option
- **Cons**
 - Loss of information
 - May need a lot of storage to memorize it
 - Tuning is tricky

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

44

Sometimes you are **forced** to use empirical distributions, simply because there is **no alternative**. In this case, you must be aware of censored data. Your data will certainly be censored, and it is customary to add an **exponential tail** to the right to take into account the probability to obtain **larger values**.

In case, the approach is the following: you take a trace, and

- a) you find an EPMF (first problem: how to choose the histogram buckets)
- b) You compute the probabilities
- c) You generate the same input with the same probabilities

If you do this, you may end up **losing a lot of information**. For instance, correlation between successive records (which is **very high** in MPEG videos, due to the GOP structure) would be lost, and your traffic would never look like an MPEG video.

Always be wary of empirical distributions, and use them if no other options are available.

Question: if you have a trace, why should you map it to an empirical distribution? because the trace may not be long enough to perform your simulation; because the trace refers to that specific historical data and can hardly be generalized.

Generating random values from a distribution

- When using self-driven simulation, you need to **generate random values** from a given distribution
 - e.g., you need to find out the arrival time of the next packet
- Not only for input generation
 - e.g., decide whether a transmitted packet is correctly received at the destination or not
- 2-step solution:
 1. **Random number generation**: generate a "random" variable uniformly distributed within $[0,1]$
 2. **Random variate generation**: transform the above variable in order to meet the desired criteria (i.e., distribution property)

In any case, a self-driven approach relies on **generating random numbers** according to a certain distribution.

Random number generation: an example

Midsquare method

1. Start with a 4-digit positive integer number Z_0
2. Square it to obtain a (up-to) 8-digit integer
3. Take the middle 4 digits, and call it Z_1
4. Place a decimal point at the left of Z_1
5. Repeat from 1 considering Z_1 as starting number

- Looks good, but...
 - Quickly degenerates to 0 (and will stay there forever)
 - Is it *actually* random?

«We are here dealing with mere “cooking recipes” for making digits»
von Neumann, 1951

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

46

Arithmetic methods to generate random numbers are sequential, with each new number being determined by one or several of its predecessors according to a fixed mathematical formula.

EXAMPLE 7.1 (from Law-Kelton). Start with a four-digit positive integer Z_0 and square it to obtain an integer with up to eight digits; if necessary, append zeros to the left to make it exactly eight digits. Take the *middle four* digits of this eight-digit number as the next four-digit number, Z_1 . Place a decimal point at the left of Z_1 to obtain the first “U(0, 1) random number,” U_1 . Then let Z_2 be the middle four digits of Z_1^2 and let U_2 be Z_2 with a decimal point at the left, and so on. Table 7.1 lists the first few Z_i 's and U_i 's for $Z_0 = 7182$ (the first four digits to the right of the decimal point in the number e).

7182 -> 51,581,124 -> 5811 -> 0.5811

5811 -> 33,767,721 -> 7677 -> 0.7611

....

In the end, you obtain a **stream** of numbers that appear random.

Intuitively the midsquare method seems to provide a good scrambling of one number to obtain the next, and so we might think that such a haphazard rule would provide a fairly good way of generating random numbers. In fact, **it does not work very well at all**. One serious problem (among others) is that it **has a strong tendency to degenerate fairly rapidly to zero**, where it will stay forever. (Continue Table 7.1 for just a few more steps, or try Z_0 : 1009, the first four digits from the Rand Corporation tables.) This illustrates the danger in assuming that a good random-number generator will always be obtained by doing something strange and nefarious to one number to obtain the next.

A more fundamental objection to the midsquare method is that it is not “random” at all, in the sense of being unpredictable. **This objection applies to all arithmetic generators.**

However, as von Neumann said “Any one who considers arithmetical methods of producing random digits is, of course, in a state of sin. [...] there is no such thing as a random number—there are only methods to produce random numbers, and a strict arithmetic procedure of course is not such a method. . . . We are here dealing with mere “cooking recipes” for making digits. . . .»

Random (?) number generation

- The truth is...

...you cannot generate random numbers!

- The second truth is...

...you do not need random numbers as such!

Let us set something straight right from the start: it is impossible to generate random numbers on a PC. If someone tells you differently (or, worse yet, sells you some code that is supposed to actually generate random numbers), just doubt it.

In any case, you **don't need random numbers** as such. You need sequences of numbers that **appear** random, and are:

- Distributed like the RV that you want to generate
- Highly uncorrelated, so that the IID assumption is reasonable.

Such numbers can be generated using perfectly **deterministic** algorithms, which will be shown hereafter.

Desirable attributes of a RNG

In fact, we need something that:

1. Generates numbers in $[0,1)$ that are
 - *Uniformly distributed*
 - *Uncorrelated*
2. Is fast and parsimonious with memory resources
3. Yields **reproducible results**
 - *Debugging*
 - *Fair comparison of alternatives*
 - *The very essence of science*
4. Provides for “separate streams” of random numbers (more on this point later on)

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

48

-Point 1 is fundamental. An RNG that lacks this characteristic is **never ever to be used**. The results that you get by using it are **totally unreliable**. A sequence like 0 0.1 0.2 0.3 0.4 .. 0.9 is uniformly distributed, but it shows a trend hence you cannot use it as RNG stream

-Point 2 is obvious. A simulator has to be as efficient as possible, and the RNG should **not be the bottleneck**. Moreover, you will not need just **one**, but **many RNGs** (we'll see this later on).

-Point 3 is what perplexes most of the students. I don't need **random numbers**. I need **streams of numbers** that exhibit the **same statistical characteristics as random numbers**, but are **generated using deterministic algorithms**:

- You can't **debug** your code unless you are able to **reproduce** the exact scenario that led you to firing the bug. Thus, you need a deterministic path that leads you there.
- Simulation is often used to **compare alternative design choices**. A comparison might be **unfair** if the two alternative competitors weren't given exactly the same input. Again, you need deterministic input to do that
- The very definition of **science** involves **repeatable experiments**. Let's see if you obtain extremely good results from an experiment, but you can't say how you obtained it because you can't reproduce the same conditions. What is not repeatable is not science.

-Point 4 is somewhat delicate, and will be dealt with later on. We can think of the different streams as being separate and independent generators, so that we can “dedicate” a particular stream to a particular source of randomness in the simulation

“Pseudo-random” number generation

In the most general case:

$$x_n = f(x_{n-1}, x_{n-2}, \dots, x_0)$$

- f is a **deterministic** function
 - Given f and $x_{n-1}, x_{n-2}, \dots, x_0$, you **always** obtain the same sequence
- $x_{n-1}, x_{n-2}, \dots, x_0$ are called the initial seeds of a sequence
 - Quite often $n = 1$, so you end up needing **only one initial seed** x_0
- Such function produces **pseudo-random numbers**
 - *Look like* random numbers
 - Same sequence can be repeated *if needed*

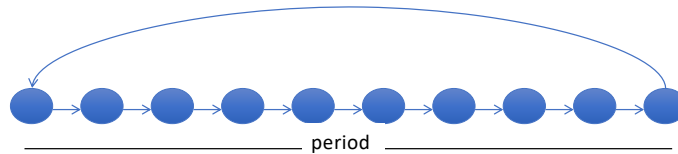
Deterministic function: same output with the same inputs.

You choose a function f **with memory**, meaning that each output is a function of the previous outputs (as in the midsquare method we saw earlier)

Most of the times, it is $n=1$, so you only need **one initial seed x_0** .

How to choose a good function f ?

$$x_n = f(x_{n-1}, x_{n-2}, \dots, x_0)$$



- Subsequent values should be uncorrelated and uniformly distributed
- Efficiently computable
- Period should be large

“A complex set of operations leads to random results” **BEWARE!**

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

50

A pseudo-random generator **ends up in a loop** sooner or later. At the very least, since it outputs results that are on n bits, it will produce at most 2^n numbers before hitting one that it has already produced. The number of iterations before you hit an “old” number is called the period of the generator. From the moment you follow the backwards arc, then **the values will not be independent anymore**, hence they should not be used. If you do a lag plot at a lag equal to the cycle, you find a straight line (perfect correlation).

The period depends:

- On the shape of the function
- On the initial seed as well.

There are mathematically rigorous tests to prove that pseudo-random numbers are uncorrelated. These are the same tests that you may use to test the hypothesis that a set of values is extracted from a given distribution (you are actually doing the same thing in both cases, if you think about it).

Beware of common folklore. The statement in the last line is **totally absurd**. “random” does not mean “I can’t figure out how you obtained this”. Complex ways to generate numbers can lead to correlated and/or non-uniformly distributed numbers.

Most programming languages have libraries or instructions to generate random numbers. **Before using them, do read the code** (or the documentation at least), or **test them** using the conformity tests that RNGs are expected to pass (using test methods that we are going to introduce later on). The fact that they are in a library does not make them suitable for any scientifically sound purpose.

Linear Congruential Generators (LCGs)

$$x_n = (a \cdot x_{n-1} + b) \bmod m$$

where $0 < m$, $a < m$, $b < m$, $x_0 < m$

- Random numbers *appearing to be* IID $U(0,1)$ are x_0/m , x_1/m , ..., x_p/m
- $b = 0$ **multiplicative** LCG
- $b > 0$ **mixed** LCG

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

51

If you want numbers distributed in $[0,1)$ rather than in $[0,m)$, then you normalize by dividing by m .
Note that the right extreme (i.e., 1) cannot occur as an output.

P = period

EXAMPLE 7.2 . Consider the LCG defined by $m=16$, $a=5$, $b=3$, and $x_0=7$.

$X_n = (5 \cdot X_{n-1} + 3) \bmod 16$ with $X_0=7$

$X_0 = 7 \rightarrow X_1 = 6 \rightarrow X_2 = 1 \rightarrow \dots X_{15} = 4 \rightarrow \mathbf{X_{16} = 7} \rightarrow \mathbf{X_{17} = 6} \rightarrow \dots$

From $n=16$ through 31, we shall obtain exactly the same values that we did from $n=0$ through 15, and in exactly the same order.

The length of the period and the way to select the parameters change based on the value of b .

LCG – period

- **At most m**
 - Can be smaller depending on the other parameters
 - If $P = m$, then the LCG is said to have **full period**
- With **mixed LCG** ($b > 0$), a sufficient condition to achieve full period is
 - m is a (large) power of two (so that modulo operation comes for free)
 - $a = 4c + 1$, for some integer c
 - b is an odd number
- With **multiplicative LCG** ($b = 0$), choosing a power of two for m implies that the period cannot be larger than $m/4$
 - Max period is achieved with $a = 8c \pm 3$ (for some integer c) and odd initial seed

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

52

Why at most m ? Because the LCG generates numbers between 0 and $m-1$.

Why a **large** power of two? Because the period cannot be larger than m , of course.

Second condition means that a is multiple of 4 (plus 1)

The necessary and sufficient conditions for achieving full period with mixed LCGs are that:

- m and b should be relatively prime $\rightarrow$ which means that they both can be divided exactly only by 1 (this condition cannot be satisfied when $b=0$, since both m and b could be divided by m)

-All prime factors of m are factors of $a-1$ as well

-if m is a multiple of 4, then $a-1$ is a multiple of 4 as well.

LCG – selection of the initial seed

- If conditions for full period are met, then **any** initial seed will do
 - e.g., mixed LCG chosen as in the previous slide
- Otherwise, the period depends on the initial seed
- Example: multiplicative LCG

$$x_n = (5 \cdot x_{n-1}) \bmod 2^5$$

- Initial seed $x_0 = 1 \rightarrow 5, 25, 29, 17, 21, 9, 13, \mathbf{1}, 5, \dots$ (P=8)
- Initial seed $x_0 = 2 \rightarrow 10, 18, 26, \mathbf{2}, 10, \dots$ (P=4)
- Initial seed $x_0 = 0 \rightarrow \mathbf{0, 0, 0, 0, \dots}$ (P=0)

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

53

This multiplicative LCG has a maximum period equal to 8 ($=M/4$) (in fact, $a=8*1-3$). That period can be achieved only with an odd seed (e.g., 1), and not with even seeds (e.g., 2).

Few people stop to think that a multiplicative LCG will always yield a sequence of zeroes if seeded with zero.

If you are using a random number generator whose period depends on the initial seed, this should be clearly stated by those who coded it. In this case, they should provide either rules or a list of **good seeds** for that RNG.

LCG – selection of the initial seed

- Generally, it is advisable to avoid:
 - Even numbers (with multiplicative LCGs)
 - Zero (always, and especially with multiplicative LCGs)
 - Unpredictable numbers, such as clock value or Process ID
 - Unless it is for fun (e.g., coding a random game of cards), or you deliberately want to make things unrepeatable or hard to reproduce (e.g., for security-related purposes)
 - Never do this when simulating, and distrust anyone who says otherwise
- If your RNG comes with too many conditions, then you should consider using another
 - Chances of doing things wrongly unknowingly are too high

Independent streams of random numbers

- Within a simulation, you generally need several streams of random numbers
 - Packet arrival times and packet sizes
 - N users transmitting on the same channel (e.g., Ethernet LAN)
- **Never** use the same RNG for more than one thing at the same time
- Use **as many replicas** of the same RNG as required
 - Seed them using seeds **far away in the period**
- Example:
 - You need n independent streams and less than k random numbers per stream
 - Your RNG has period $P \geq nk$
 - Then, instantiate n RNGs and seed them with

$$s_i = x_{(i-1) \cdot k} \quad 1 \leq i \leq n$$

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

55

In many cases, you don't need **just one stream of pseudo-random numbers**, but you need **many** (suppose you have to simulate a cellular network with k users transmitting traffic). These should, of course, be **independent**. For this reason,

You can't use a single RNG to extract all the values for the k streams. In fact, the tests for the RNG may not be able to guarantee that "arbitrary samplings" of your RNG yield streams of uncorrelated numbers. Even if x_i and x_{i+1} are uncorrelated, x_i and x_{i+k} (with a large k) may be correlated, or they may not be uniform. If you are using the same RNG to produce k streams, you may end up believing that the single substreams have statistical properties that they do not possess in fact.

You should instead instantiate **n replicas of the same RNG**, and use **different seeds** for each replica. The seeds should be chosen based on the period p . Each seed should be a number which is k elements away, assuming that the simulation requires each RNG to generate at most k numbers.

Of course, using the **same seed** for all generators (a common mistake) is a very dumb thing to do.

More on this later on, again.

$n=3, k=4, P=12$

Independent streams of random numbers

Why using several replicas of the same RNG?

- Preserving statistical coherence
 - An arbitrary subsampling of a stream may not have the right statistical properties (uniform distribution, lack of correlation, etc.)
- Avoid (very nasty) hidden side effects
 - If you use a single RNG, changing just one parameter of a random variate changes all the other random variates as well
 - Debugging is almost impossible
 - Fair comparisons (paired experiments) are impossible

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

56

1) In fact, the tests for the RNG may not be able to guarantee that “arbitrary samplings” of your RNG yield streams of uncorrelated numbers. Even if x_i and x_{i+1} are uncorrelated, x_i and x_{i+k} (with a large k) may be correlated. If you are using the same RNG to produce k streams, you may end up believing that streams are independent when they are not.

One may counter that **good RNGs** should be exempt from this kind of problem. They may be, but then there is still a residual probability that, by subsampling (at a non-constant pace) a stream of random numbers, you do end up with something which is not uniform in $[0,1]$. Perhaps you only see “large” values, and you cannot exclude it, nor you can know it a priori (and you will never check it a posteriori, of course).

2) This is a lot more important. If you are using the same RNG for different purposes, you are introducing **dependences** between streams of RNs that should be independent of each other. Here’s an example

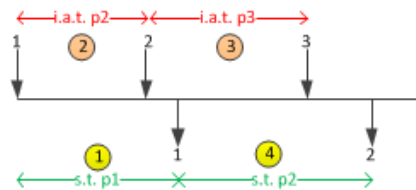
Independent streams of random numbers

Example 1

- One stream used to generate both interarrival time and service time

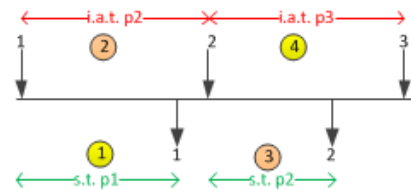
Interarrival time $\sim \exp(1/1)$

Service time $\sim \exp(1/2)$



Interarrival time $\sim \exp(1/1.5)$

Service time $\sim \exp(1/2)$



- Changing seeds makes **service times change** → unwanted side effects

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

57

Assume that you use **one** stream to generate **both** interarrival and service times:

- When a packet arrives

- 1) If the queue is empty, the packet goes under service, and its service time is generated
- 2) The next interarrival time is generated.

- When a packet departs

- 1) If the queue is non-empty, another packet goes under service, and its service time is generated

This makes debugging **very hard**, and – worse yet – makes **paired experiments impossible**. In fact, the sequence of your service times comes to depend not only on the initial seed, but also on the sequence of arrival times. You will never be able to replicate the same sequence of service times, unless you also replicate the same sequence of arrival times.

Independent streams of random numbers

Example 2

- One stream used to generate packet transmission by N users
- Assume that the RNG produces the following stream: 7, 3, 12, 1, 5, 9, 6, ...

• $N = 2$

• $N = 3$

- Adding one more user **changes the behavior of the other users** → unwanted side effects

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

58

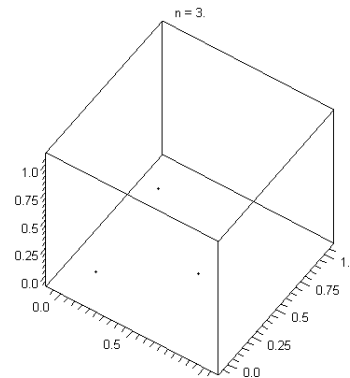
The N users taking turns at picking numbers from the same random stream.
Adding one more user affects the choice of other users.

Known properties/problems of LCGs

- Every x_i can be computed **deterministically**

$$x_i = \left(a^i x_0 + \frac{b(a^i - 1)}{a - 1} \right) \bmod m$$

- x_i can only take *rational* values
 - $0, 1/m, 2/m, 3/m, \dots, P/m$
 - No chance of getting, say, $\frac{0.8}{m}$
- Many sequences of numbers will never come out



Source: Wikipedia

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

59

This works for both multiplicative and mixed LCGs.

Note that the fact that you can compute x_i deterministically can be put to good use. You can figure out **values far away** and use them to seed multiple instances of the generator as in the previous slide, **provided that** you are able to make the above computation **exactly** (it may not be easy if a , i and b are large).

If you analyze tuples of subsequent numbers from the same LCG, e.g., by plotting vector $[X(i), X(i+1), X(i+2)]$ in a 3D space, you observe that points tend to align on a **finite number of hyperplanes**. Depending on the LCG parameters, the number of hyperplanes can be **very small**, even with full-period mixed LCGs. This means that there are **regions of n-dimensional space** that are never explored by this LCG.

In other words, there are **many sequences** that will never come out. This is bad, because you are missing out on a large portion of space.

C++ rand() function

- C/C++ provides two functions, defined in `stdlib.h`

`int rand()` generates a number between 0 and `RAND_MAX` (included)

`void srand(unsigned seed)` seeds the RNG

- The RNG algorithm used by the above functions is **compiler-specific** (see wiki page for LCG)
- Example of `rand()` code for a compiler:

```
return ( (ptd->_holdrand = ptd->_holdrand *  
        214013L + 2531011L) >> 16) & 0x7fff );
```

C++ has a random generator, which is called using the **rand()** function.

There is **no specification of an RNG algorithm**, so you had better assume that:

- The same code will produce different streams if compiled on two machines, even with the same seed, thus being highly **non-portable**, hence may lead to different results on different computers
- There is no guarantee that the stream of numbers produced by these generator will pass any **statistical test**. In particular, the one in the slide is not even an LCG, so you don't really know its statistical properties.
- You can't have different, **independent streams** of pseudo-random numbers.

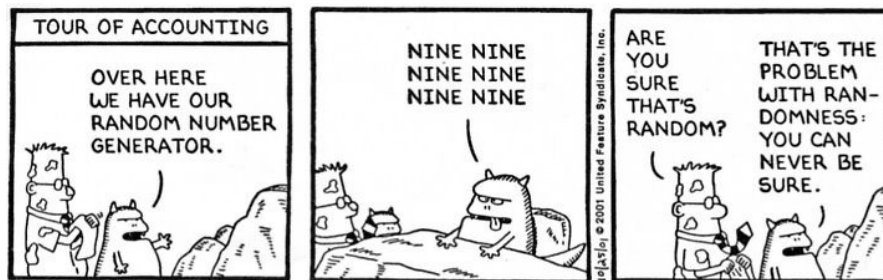
Bottom line: **never** use this in simulations.

Other RNG algorithms

- Quadratic congruential generator $x_n = (ax_{n-1}^2 + bx_{n-1} + c) \bmod m$
- Multiplicative recursive generator $x_n = (a_1x_{n-1} + a_2x_{n-2} + \dots + a_qx_{n-q}) \bmod m$
- Tausworthe generators
 - Operate directly on bits. Next bit is selected as random
 - Very large periods
 - Related to cryptographic methods
- Mersenne-Twister generator
 - Huge periods ($2^{19937} - 1$)
 - Memory-hungry, ~2KB per instance

Memory-hungry-ness is not a big problem nowadays. However, take into account that you may need **many** RNGs ($\sim O(1000)$), in which case it may become a problem. Compare this occupancy with the one of an LCG (which is sizeof(int)).

Testing RNGs



We observe a stream of numbers and we ask ourselves: is it truly random? How can we know that?

Brilliant citation from «Inglorious bastards»

Testing RNGs

Are drawn numbers really $U(0,1)$?

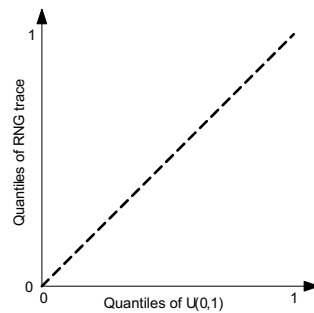
- Need to check that the RNG
 - Produces a stream of **uniform** numbers in the **range [0,1)**
 - Produces **uncorrelated** numbers
- Several techniques to test the above conditions

If you are **proposing** an RNG algorithm yourself (which is unlikely) or you are **testing** an existing RNG which you found in someone else's code (which is more likely), here is what you should do. You should test that the RNG:

- a) Produces a stream of **uniform numbers in [0,1)**, and
- b) That those numbers are **uncorrelated**.

Testing RNGs

1 – Run one long sequence and check *visually* using a **QQ plot**



- You should obtain a straight line – check for deviations, especially at the tails

Let us start with the first property.

The first test that you use on an RNG should ascertain whether the RNG actually produces numbers that are $U(0,1)$. This can be done in two ways:

- Visual tests, such as a QQ-plot (quantile vs quantile plot)
- Numerical tests, such as a chi-square test.

Both are particularly straightforward when testing for a uniform probability.

Testing RNGs

2 – Do a **chi-square test** at a level $1 - \alpha = 0.9$

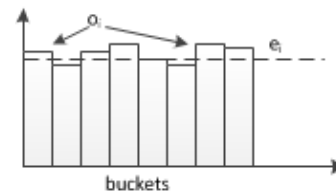
- Draw large sample n , divide it into k buckets of equal width
- Bucket i contains o_i elements, expected $e_i = \frac{n}{k}$
- Compute the following number

$$D = \sum_{i=1}^k \frac{(e_i - o_i)^2}{e_i}$$

- If $D > \chi_{\alpha, k-1}^2$, then **reject** the $U(0,1)$ hypothesis

- How to choose k ?

- Such that $\frac{n}{k} > 10$



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

65

Basically, you check how far the number of elements in each bucket is from the expected number

The chi-square test can be used against **any** distribution (not just the uniform).

Its result depends a lot on how you select the number of buckets, their width (which need not be equal), and the confidence level. Always be wary of these tests (prefer QQ plot when in doubt).

The differences should be **normal**: the further away the deviation from the expected value, the less likely. If you sum several **squared normals** you obtain a Chi-square. Therefore, if D is an **unlikely value** for a chi-square, you should reject the hypothesis.

It can be extended to n dimensions. If you want to make a 2-dimensional test, you take each **couple of random numbers**, x, y , and put it into any of the k^2 buckets that you have prepared. Each of these buckets should include $e(i,j) = n/(k^2)$ observations at the end, etc. etc.

The Chi-squared test works with **large samples** ($\sim O(1000)$). Otherwise the test is unreliable.

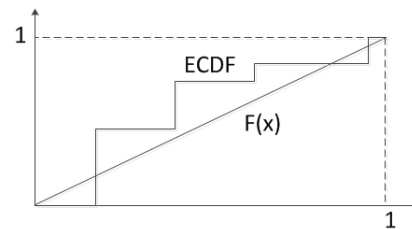
Testing RNGs

3 – Kolmogorov-Smirnov test

- Draw a (not too large) sample n and compute ordered stats $X(i)$
- Compute the following numbers

$$D^- = \sqrt{n} \cdot \max_i \left(\frac{i}{n} - X_{(i)} \right)$$

$$D^+ = \sqrt{n} \cdot \max_i \left(X_{(i)} - \frac{i-1}{n} \right)$$



- Test D^- and D^+ against the *KS* distribution
- If either is larger, **reject** the $U(0,1)$ hypothesis

This test amounts to finding the **maximum** vertical distance between a theoretical distribution (in this case, a uniform) and the ECDF of your data (in this case, the O.S.s of your RNG). The two values D^- , D^+ are in fact these maximum distances (times a coefficient).

If these distances are **small enough**, then the assumption of matching cannot be rejected. The test should be done against a particular distribution, called the K.S. distribution, which you can find on tables. This depends on the number of samples and the required confidence.

This test can be used with pretty much any distribution. It works better with small samples (when the chi-square test would instead be less significant).

Testing RNGs

Are values independent?

- Visual inspection
 - Use **lag plots** (i.e., plot x_i, x_{i+k} for various k)
 - See if graphs show any trend – if they do, numbers are correlated
- Numerically
 - Plot a correlogram

Now, a sequence of n values equal to $1/n, 2/n, 3/n, \dots$ is perfectly uniform. It would pass the chi-square test at any significance level. However, it is not to be mistaken for a pseudo-random sequence.

The reason is that subsequent numbers are **correlated** (positively so, in this case). Correlation can be spotted easily in a **lag plot**. Lag plots should be done for several lags.

Correlogram (aka Auto-correlation function plot) is a visual way to show serial correlation in data that changes over time (i.e. time series data).

Testing RNGs

Many other tests are possible

- Experimental tests (test the sequence)
 - Chi-square in k dimensions ($k = 2, 3, \dots$)
 - ...
- Theoretical tests (test the algorithm)
 - Spectral test
- Refer to the literature

Underlying message: **never ever use an untested RNG!**

Random variate generation

- Given a **reliable** stream of uniform (pseudo-)random numbers, we can compute values for arbitrary random distributions
- Unfortunately, there is **not** a general approach that can be followed for **any** distribution
- Some common methods
 - Inverse transform
 - Convolution

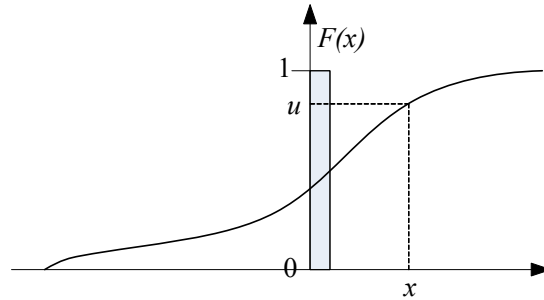
So far, we have learned how to obtain a **reliable** stream of numbers, uniformly distributed in the range $(0,1)$

Random variate generation – inverse transform

- Suppose you need a sample x from a RV with a given cumulative distribution function $F(x)$ that is continuous and strictly increasing
- Let F^{-1} denote the inverse function of F

- The method of *inverse transform* is

1. Generate $u \sim U[0,1)$
2. Return $x = F^{-1}(u)$



- Can use this method if you are able to obtain F^{-1}

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

70

This is one way to generate RV with a given distribution. It is not the only one, of course.

It is quite clever and “obvious”: the codomain of a CDF is $[0,1]$, so if I generate a RN which is $\sim U(0,1)$ I can map it to a value of the domain.

Random variate generation – inverse transform

Example: suppose you want samples from an exponentially distributed RV with mean $1/\lambda$

- its PDF is: $f(x) = \lambda \cdot e^{-\lambda x}$
- Its CDF is: $F(x) = 1 - e^{-\lambda x}$

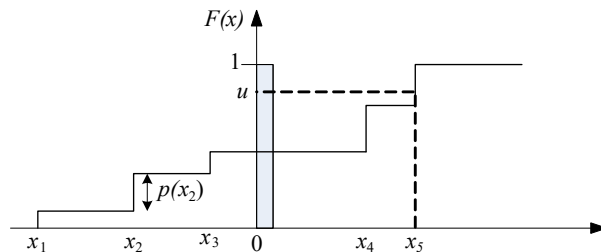
• Apply the method of inverse transform:

1. Take a random variable u uniformly distributed within $[0,1)$
2. Invert the CDF and obtain

$$x = -\frac{1}{\lambda} \cdot \ln(1 - u)$$

Random variate generation – inverse transform

- With discrete RVs, the inverse transform has a very intuitive explanation



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

72

With discrete RVs, interval $[0,1)$ is divided into subintervals which are as large as the values of the PMF. Assume that I extract a $U(0,1)$ RV, and the value corresponds to the step of x_5 : this means that I have extracted value x_5 according to that discrete RV.

Obviously, when transforming for generic **discrete variates**, especially those with a large number of values, we need to do things **efficiently**. At the very least, a logarithmic search is due.

A discrete RV can always be represented with an n -vector of pairs $\{x, f(x)\}$, sorted by increasing x .

Random variate generation – convolution

- Sometimes, we are unable to write down F^{-1}
- For instance, the Normal distribution $N(\mu, \sigma^2)$ has no closed-form CDF
- *Central Limit Theorem*: the sample mean or sum of a sufficiently large number of IID RVs with finite mean and variance is approximately normally distributed
- The *convolution* method is:
 1. Generate $Y_1, Y_2, \dots, Y_m \sim U(0,1)$
 2. Compute $X = Y_1 + Y_2 + \dots + Y_m \quad \rightarrow \quad X \sim N(m/2, m/12)$

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

73

The **inverse-transform method** does not work with all the RVs that are somewhat **related to the Normal distribution** (including the normal itself). Notably, it does not work when the **CDF is not known** in a closed form (hence it can't be inverted).

In order to generate Normal variables, you can resort to the **Central Limit Theorem**. You generate a high number of IID Uniform RVs, and their sum will be distributed as a Normal variable. This is called the convolution method, since the PDF of the sum of IID RVs is in fact the convolution of the single PDFs.

In general, you need some IID RVs (at least 6-7). This method has two drawbacks:

- a) It is costly, in any case. You need to use 6-7 good draws of an RNG to produce just one normal.
- b) You introduce errors: you will never observe large-modulus values, since uniforms are finite. If you want very large values, you need to waste very many seeds.

You obtain X normally distributed with mean $m/2$ and variance $m/12$.

You can then normalize it to $Z \sim N(0,1)$ as $Z = (X - \text{mean}) / \text{sqrt}(\text{variance})$. If you want a custom mean' and variance', you can then rescale and shift Z as $Z' = \text{mean}' + \text{sqrt}(\text{variance}') * Z$

Note that you might, in theory, use the standard approximation for the Normal percentiles in order to invert the Normal. However, this introduces **errors**, especially at the tails, so you can't risk it. One thing is to use it for (approximate) visual plots, another is to generate good-quality Random Variates.

Random variate generation – practical cases

- Continuous RVs

- $U(a,b)$

- Generate $U \sim U(0,1)$
- Return $a+(b-a)*U$

- $Exponential(\lambda)$

- Generate $U \sim U(0,1)$
- Return $-\ln(1-U)/\lambda$

- $Weibull(a,b)$

- Generate $U \sim U(0,1)$
- Return $b*(-\ln(U))^{1/a}$

- $Pareto(a)$

- Generate $U \sim U(0,1)$
- Return $1/U^{1/a}$

- Discrete RVs

- $U(a,b)$

- Generate $U \sim U(0,1)$
- Return $a+\text{floor}((b-a+1)*U)$

- $Bernoulli(p)$

- Generate $U \sim U(0,1)$
- Return $(U \leq p)$

- $Geometric(p)$

- Generate $E \sim \text{Exp}(-\ln(1-p))$
- Return $\text{floor}(E)$

- $Binomial(n,p)$

- Generate n Bernoulli(p)
- Return sum

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

74

More examples can be found in the Law-Kelton book.

Note that generation of random variates must be **fast**. If you write down code for this, it should be highly optimized: perform all the operations involving constants **once**, and store the results.

The explanation for the geometric comes from the known relationship between the exponential and the geometric.

Note that generating a binomial the way it is explained here is **inefficient**, especially for a large n , so another method is commonly used.

Collecting statistics in the simulator

- How to compute the *average* queueing delay?

$$D = \frac{\sum_{i=1}^M delay_i}{M} \quad \text{where} \quad delay_i = departure_time_i - arrival_time_i$$

- In order to compute D, you can either:
 - Write down all the events on a **log file**, then parse it offline and make computations
 - Associate an arrival time variable to each queued packet, compute the delay of the packet at departure time, and **maintain an ongoing sum** online. At the end of the simulation, compute the average

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

76

[Recall the simulator flowchart of a few slides ago]

A simulator processes events in order to compute some output numbers. The part devoted to computing the measures of interest is called **statistics collection**. Let us review what fundamental choices we have to make on that part.

Assume that we want the average delay of a flow of packets as a result. There are **two (extreme) strategies to obtain it**:

- a) Log all the events in the simulation (related to packet arrival/departure etc.) on a **log file**. After the simulation terminates, take the trace file and parse it, thus computing the sample mean of the delay
- b) Store all the information you need in **internal data structures** (those that we called “statistics gathering” some slides ago), and compute the final result only. In this case, it is enough that you use a) a counter, which stores the sum, and b) a counter which is increased at every packet departure.

These are the two extremes, but anything in between is possible. For instance, you may want to use a log file, and log the delay of each packet (instead of logging arrival/departures etc.). Or you may want to store the event trace in memory, and then dump it all on the disk only at the end of the simulation. What are the pros/cons of each approach?

- A) Longer execution times: **disk writes cost a lot** On the other hand, you get **much more information** from a complete log file. For instance, I might want to know the **sample variance** of the delay, and I would be able to compute it from the tracefile without i) modifying the simulator, and ii) running the same simulation again. A tracefile gives you a lot of information which is useful during the **debugging phase**.
- B) Shorter execution times (often considerably so), less flexibility (you can only get out of the simulator what you programmed it for).

The optimal choice depends, thus, on your objectives. Event logging is normally most useful during the debug phase, and should be disabled afterwards (unless you have good cause to do otherwise).

Example – using log files

- For each served packet, there is one line in the log file

```
time 0: packet 1 arrives
time 0: packet 1 goes under service
time 7: packet 2 arrives
time 10: packet 1 leaves
time 10: packet 2 goes under service
time 12: packet 3 arrives
time 16: packet 4 arrives
time 20: packet 2 leaves
time 20: packet 3 goes under service
time 30: packet 3 leaves
time 30: packet 4 goes under service
time 40: packet 4 leaves
```

$$D = ((10-0) + (20-7) + (30-12) + (40-16)) / 4 = 16.25$$

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

77

In C++, you would need to open a stream to an output file and write strings to that.

Note that this approach allows you to compute other quantities, e.g.

-Buffer occupancy

-Throughput

-Jitter

-...

In order to parse these files, you should resort to:

-Scripting languages (Perl, Python, etc.)

-Ad-hoc existing tools (gawk, pandas)

-Home-brewed C code (strongly discouraged: it takes a lot of time, it is highly inflexible, and error prone).

Example – online computation

- Add ad-hoc lines of code in the relevant locations in the code

```
// packet i arrives
arrival_t[i] = <get current time>
...

// packet i leaves
sum_delay += <get current time> - arrival_t[i];
served_pkts++;
...

// simulation ends
D = sum_delay / served_pkts;
```

Note that these chunks of code will be located in different event handlers.

Averaging quantities

- Several types of metrics can be extracted from a simulator
- Most of them are **averages of quantities** measured during the simulation

However...

There are **several types of averages**



Different data structures involved in maintaining them

There are **several types of averages**, depending on how you analyze the data sampled during the simulation

Metrics – Rate

- Given N samples $\{x_1, x_2, \dots, x_N\}$, the value of a **rate** metric is

$$y = \frac{\sum_{i=1}^N x_i}{\tau}$$

where τ is the **time interval** during which the samples were collected

- Example:
 - x_i is the size of the i -th packet served by a server/router, in *bytes*
 - τ is the time interval under observation, in *seconds*
 - y is the **throughput** of the server/router (in *bytes/s*)

Data structures:

- Sum of the samples
- Start time
- End time (last two items could be ignored if you consider the entire simulation time, just use the simulation clock variable)

Metrics – Time-independent samples

- Given N samples $\{x_1, x_2, \dots, x_N\}$, the value of their **sample mean** is

$$y = \frac{\sum_{i=1}^N x_i}{N}$$

- Note that y *does not* depend on the time when samples were collected
- Such samples are also called **impulses**
- Example:
 - x_i is the delay of the i -th packet served by a server/router, in *seconds*
 - N is the total number of packets served by the server/router
 - y is the **sample mean** of the delay, in *seconds*

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

81

“impulses” are values that are meaningful at the moment they are generated, like delays.
If you change the time instant t_j of observation j , then nothing happens to y .

Data structures:

- Sum of the samples
- Number of samples

Metrics – Time-dependent samples

- Average number of students during a class?

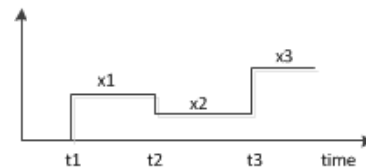
In this example, the average number of students changes depending on the time they enter/leave the class.
We need to consider how the status of the system holds over time

Metrics – Time-dependent samples

- Given N time-dependent samples $\{(x_1, t_1), (x_2, t_2), \dots, (x_N, t_N)\}$, the value of their **mean** is

$$y = \frac{\sum_{i=1}^N x_i \cdot (t_{i+1} - t_i)}{\sum_{i=1}^N (t_{i+1} - t_i)} = \frac{\sum_{i=1}^N x_i \cdot (t_{i+1} - t_i)}{t_{N+1} - t_1}$$

- Such samples are also called **levels**
- Example:
 - x_i is the length of the queue after i -th packet has arrived, at time t_i
 - y is the **average queue length** during period $t_{N+1} - t_1$



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

83

These are not impulses, they are **levels**: values that are meaningful **until they change**.
If t_2 becomes $t_2 + \delta$, then y will be different

In our single-queue example, this type of metric can be used to compute the average number of packets in the queue.

Can see this as a **weighted average**, where the time interval is the weight of the sample and the denominator is the sum of the weights.

Data structures:

- Each and every sample, along with its duration

Not only averages

“What is the probability that the occupancy of the buffer grows over 100KB?”

“What is the 99th percentile of the delay?”

- Which means: *“what is the largest delay obtained, provided that I am not interested in those delays that only happened in 1% of cases?”*

- In these cases, we need to estimate the **distribution** of a metric, instead of its average value

Warning!

Estimating distributions is a very complex task... *unless* we can tolerate a few approximations (spoiler: we can)

- Can we obtain the **PDF** of a metric from a finite number of samples?

Since the PDF of a random variable is a continuous function, you would need infinite samples
So, we need to find an approximation.

Metrics – Distribution

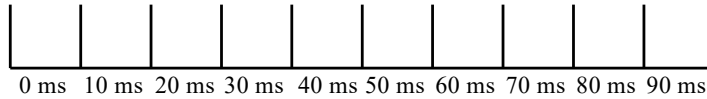
Example: distribution of the delay of packets in the queue of a server or a router output interface

- **Step 1:** decide the **range** of interest of your metric
 - Assuming that packets are discarded after 100ms, then the range *[0ms, 100ms]* is safe
- **Step 2:** decide the **granularity** that you can accept
 - We assume *10ms*
- **Step 3:** divide the range of interest in equal-sized **bins** (or **buckets**). The size of bins is the granularity we have chosen earlier

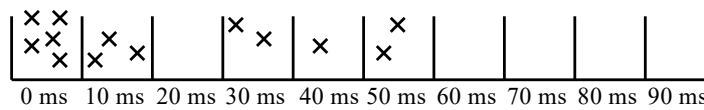
Q: what is the probability that a packet experiences a delay less than 20ms?

Note the difficulty of this task: you need some knowledge of the system you want to evaluate in order to select the right range and granularity.

Metrics – Distribution

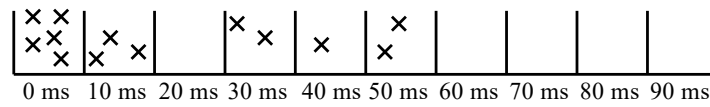


- Whenever a sample is collected during the simulation, **add 1 to the corresponding bin**
 - Keep track of samples overflowing the range, if any

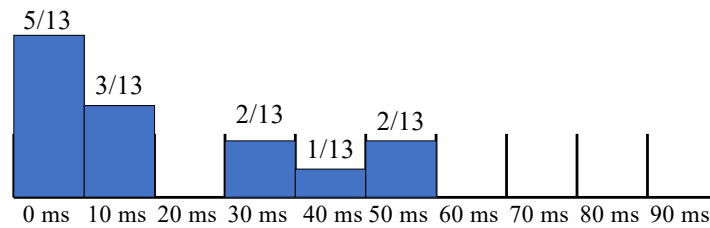


At the end of the simulation, you should **count** the number of observations that are in the “overflow” bin. If they are few, then your selection of extremes for the measurement interval is correct, otherwise you may want to select a larger interval and start again.

Metrics – Distribution



- At the end of the simulation, derive an estimate of the **Probability Mass Function (PMF)**



- From here, it is also possible to derive an estimate of the **Cumulative Distribution Function (CDF)** or **quantiles**, if needed

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

87

Probability mass function = discrete probability density function

Q: what is the probability that a packet experiences a delay less than 20ms? $5/13 + 3/13$

Note that the **size of the bins** determines the way data will look.

-Too large: lack of accuracy and details (very different samples end up in the same bin)

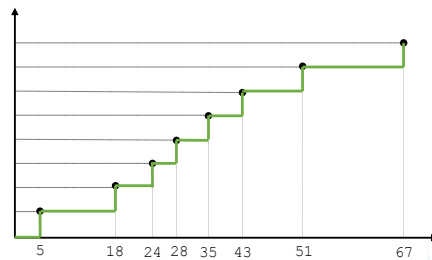
-Too small: lack of resolution: you end up having mostly **empty** bins or bins with **one or few** samples within. This does not tell you anything about the real shape of the pmf/cdf

Data structures:

- Array of integers (whose size depends on the number of bins)
- Counter for overflow

Metrics – Distribution

- If you just need the **CDF** or **quantiles** of your metric (no PDF/PMF), you do not need to define ranges & bins
- Use the actual samples you extracted (can be stored using just one array)
- Example: the simulation produced $N=8$ samples of the delay of packets



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

88

Data structures:

- Array including the samples

How to do a simulation analysis

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

«Simulation analysis» as all the steps that are involved in the workflow:

- Defining the objectives and KPIs
- Modeling and model validation
- Defining the factors
- Selection of the appropriate tool
- Implementation and verification
- Calibration
- Design of experiments
- Running simulations
- Data collection and analysis

Simulation workflow

- Simulation, as any other engineering practice, has a **workflow**
- No workflow → no science
- Simulation workflow
 - Set of actions to be taken
 - Importance (very) often misunderstood
 - Automation of some steps possible
- Much like software engineering, with some **added complications**

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

From this slide on, we check the list of everything that is required when you make a simulation analysis. Take the following perspective: you are leading a **large-scale simulation project**, coordinating several scientists and engineers, to do **something important** for a **big customer**. The checklist items will be shown **in a sequence**. However, the real process often requires jumping forwards and (mostly) backwards. Every time that, at a given stage, you feel that something is wrong, you just step back and set it right.

The importance of managing the workflow correctly is normally **underestimated**, in both industrial and scientific environments. Most people think that once you have written a bug-free simulator (given a system model), what you feed it as an input, how you analyze its output, etc. are not particularly important aspects, as if the code itself would be able to guarantee the correctness of its usage.

This is, however, plainly wrong. If you concentrate on the code and neglect all the rest, you don't just risk failing, you are simply **certain that you will**. And all the time spent in honing the code to perfection will be – of course – wasted, since the results will not have anything to do with reality. Most computer engineers/scientists do not possess the background to **appreciate the importance** of what lies **around the simulator**.

Now, some of the steps of the simulation workflow are **boring as hell**, especially if you are expected to do them manually: file comparisons, computations, etc. You shouldn't be surprised to learn that these are done **manually most of the time**. This makes simulation analysis **costly and error prone**. Every time you have long, boring tasks at hand, the obvious solution is **to write some code that automates those tasks**. There are some solutions, but the field is largely unexplored.

Simulation, in a sense, is not too different from software engineering. If you need a 200-LOC program for your own purposes, just write it any way you wish. If, instead, you do **programming in the large**, then you can't improvise: you have to follow standards, best practices, etc. (otherwise you won't be able to obtain state-of-the-art results).

Similarly, if you have to write a 200-LOC simulator for your own purposes (e.g., a quick-and-dirty evaluation of two alternatives in a simplified setting), just do it (and never think of using its results for anything else). If you have to evaluate whether to buy equipment A or B, and your company is investing some ME on equipment, your analysis can't be anything less than state-of-the art.

Why bother?

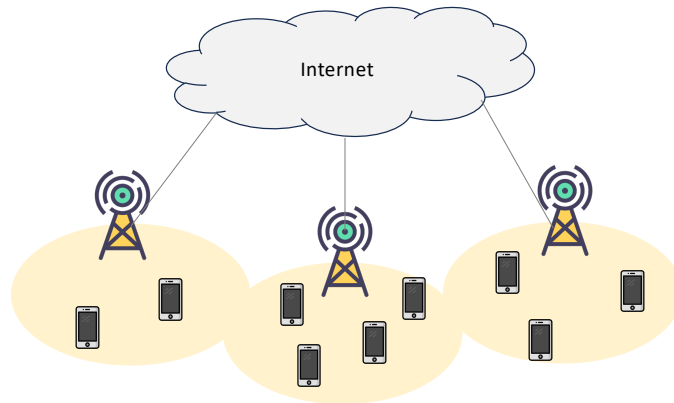
- Simulation is used in many **safety-critical** environments
 - Military, first and foremost
 - Environmental protection
 - Aircraft and satellite navigation
- It is used by **very large** companies to support strategic decisions
 - Millions of euros involved

While the importance of designing good applications that need to go to the market is clear, sometimes the importance of designing a good simulation analysis is not fully understood. However, results and decisions coming out from a simulation analysis can have great impact on the specific environment it is designed for, like safety-critical ones or where a lot of money is involved.

In some of these fields, prototyping shouldn't be used. And, of course, you can't improvise.

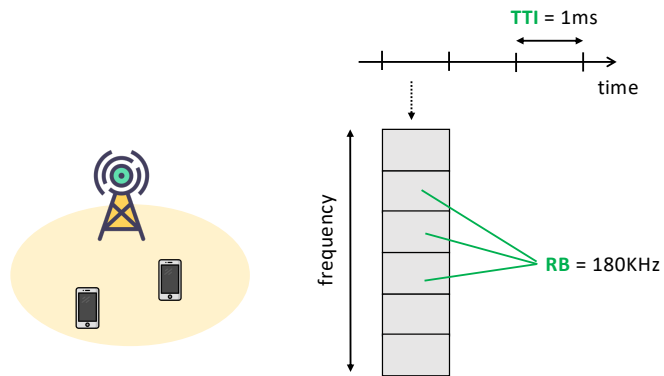
Case study: VoIP in 5G networks (1/4)

- User Equipments (UEs) establish a **Voice-over-IP** communication with their respective remote endpoint
- Final user's **Quality of Experience** is determined by
 - Throughput
 - End-to-end latency
 - Jitter
 - Packet loss rate
 - ...



Case study: VoIP in 5G networks (2/4)

- The 5G Radio Access Network plays a big role in determining the final performance
- Every **T**ransmission **T**ime **I**nterval (**TTI**), each **base station** allocates mutually exclusive **R**esource **B**locks to their UEs based on their **channel quality**



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

93

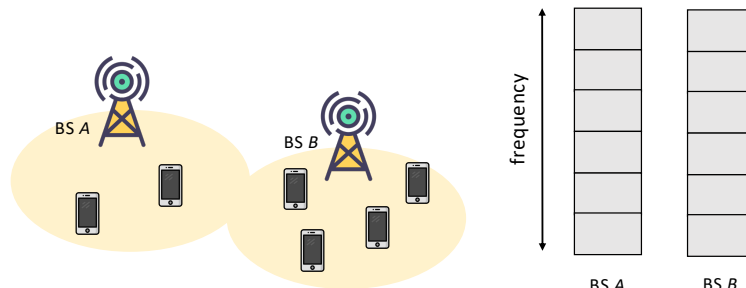
Traffic to be sent to UEs is queued at the base station. Periodically, the base station performs scheduling by allocating transmissions into Resource Blocks based on a) the channel quality indicator received by the UEs, and b) the type and amount of data sitting in the UE queues at the base station

The channel quality indicator (CQI) determines how many bytes can be fit into one RB

Case study: VoIP in 5G networks (3/4)

Problem

- Neighboring base stations use the **same** frequency spectrum
- Transmissions from neighboring cells occurring in the same RB at the same TTI produce **interference**
- **Performance degradation**, as UEs:
 - perceive worse channel quality → need more RBs → base station has less total capacity



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

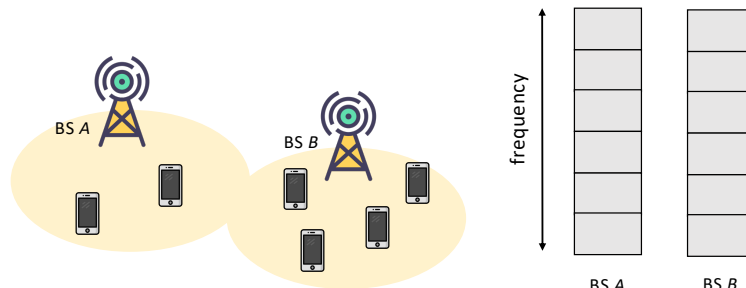
94

Base stations use the **same frequencies**. This means that, unless they coordinate, they will interfere with each other's UEs, especially if they are nearby

Case study: VoIP in 5G networks (4/4)

Possible solution

- Use **inter-cell interference mitigation** techniques
- Base stations **coordinate** to decide *who* uses *which* RB frequency *when*



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

95

Coordination means that Base Stations agree first on who is going to use which frequencies, so as to mitigate interference.

This has a threefold benefit:

- The UEs' channel quality indicator on coordinated RBs increases
- The same data can be inserted into **fewer RBs**, thus freeing resources to serve more users simultaneously
- Less power is consumed (operators pay the bill for the power they consume, much like we do)

The simulation workflow – objectives

Define the objectives

- Choose a SMART (Specific, Measurable, Achievable, Relevant and Time-bound) objective
 - For instance, assess the effectiveness of a video-streaming protocol
 - “I want to simulate the 5G network” is not an objective
- SMART objectives help you plan the necessary **time, manpower, skills** for the project
- In our case study, the objective is to study the benefits of different 5G network’s interference mitigation techniques on VoIP traffic performance

Trivial as it may seem, this is fundamental. Never start to write down a simulator **before** knowing what you want to do with it.

- Specific as in “not vague”: it must help you clearly identify *what* has to be done in order to complete your analysis
- Measurable as in “we can tell when we’re finished”: you must have means to *quantify* the objective you set
- Achievable as in “it is something that really *can* be done with the time, money and expertise we have at our disposal”
- Relevant as in “it must make sense to pursue this objective”
- Time-bound as in “it is something that provides you a clear deadline; the task has an ending”

Note that the quoted statement fails at all the requirements, yet is what most people say when they embark on a simulation project.

-There is no achievable goal. You will never know if you’re finished (when you’re bored, most likely)

-It is unspecific: one thing is to assess the performance of **web browsing** on that technology, another is to assess its **bit error rate**. You need very different skills to assess the two things, and very different models

- In the first case, you can probably model the channel as something which gives you an error probability (a Markov chain will do most of the times). On the other hand, you will need the full protocol stack, with good models of user behavior, of a web server, etc.
- In the second case, you need a very accurate model of signal propagation at a given frequency, given the power level of the transmitter, the coding scheme, the antenna layout, etc. In this case, all the models required in the previous case are unnecessary.

The simulation workflow – indices

Define the performance indices

- Depend on the objectives
- For instance, we would like to show that our video-streaming protocol is *better* than the existing ones, like DASH or HLS
- In our case study, *better* means “it achieves a higher end-to-end throughput and/or a lower latency”

This is trivial as well (but needs to be stated nonetheless): once you have a high-level objective, you need to translate it into **measurable performance indices**.

At the end of the day, this will define the statistics that your simulation software will produce

The simulation workflow – modeling (1/2)

Build a model of the system

- Abstract away some of the details
- Include possibly many sub-models
- Example:
 - Router/base station sub-model (processing delay? Output vs. input queueing? CRC/checksum?)
 - Link sub-model (propagation delay? Bit corruption?)
 - Encoding/decoding sub-modules (computation delay? Buffer sizes?)
- Maintain an **assumption document** stating which simplifications were made and why
 - State controversies for future review

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

98

Once objectives are defined, the next step involves creating models that abstract away some of the complexity of the real system. The models must retain enough detail to allow for the goals of the simulation study to be met.

Modeling is a complex activity; one that requires a good dose of insight and a fair amount of experimentation.

You need to **know deeply** that piece of reality, possibly enlisting the help of someone else (your customer, for instance). Most of the times, you can only set up a simulation study by **interacting with the users**. If you are hired to simulate a manufacturing plant, chances are that you know nothing about either manufacturing or plants. In this case, modeling is by necessity a **joint effort** with the users (likely, the owners of the plant, the workers, the technicians, etc.). Whenever there's interaction, there must be **documentation**. It is also very helpful to **cover your back** later on.

Modeling is really an **art**. Much like playing music or writing songs. The more you do it, the better you become at it. Unless you're hopelessly tone-deaf, of course.

The simulation workflow – modeling (2/2)

Beware of common folklore

- “The more **detailed** the model, the fewer assumptions are required, hence the more **reliable** the model”
- Example: you want to simulate the flow of packets inside a base station, and you know that processing packets incurs some delay. You can model this delay using:
 - a fixed value
 - a probability distribution
 - a detailed model of the base station internals (memory, bus architecture, processors, caching etc.)
- Which one is preferable? Which yields the most reliable results?
- A more detailed model needs more detailed **knowledge**
 - The lack of which makes the model **unreliable**

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

99

The problem is that it is not too **difficult** to build a software replica of how a router behaves. However, you need to insert **parameters** in this model, and you may not have the slightest idea of what acceptable values for those parameters may be.

This way, you simply end up making **wrong assumptions**, which may invalidate the whole model.

If you stick to, e.g., an exponential distribution of base station traversal time (because that's what you find in the literature), with a mean time taken from vendor data sheets, you surely make **some error** (due to approximation).

If you, instead, try to model the internals of the base station without being a true expert in base station architectures, you **surely** end up forgetting something crucial (or making wild assumptions about details that you can't fathom), and may

- miss the true result, even by orders of magnitude
- Harbor the **false certainty** that your model is reliable **because it is so detailed**.

“**I don't know enough**” is by far the hardest phrase to force out of (computer) engineers.

There's another subtlety in this, and it's **upgradability**. If you are spending a lot of time in modeling something, you may want to be able to **upgrade** your model when conditions change.

In this example, you may want to assess what changes if new-generation base stations come out. These may have a completely different architecture. Perhaps you spent some time honing the fine details of the old base station architecture, and all this **effort will be wasted**. If you had stuck to a **higher-level model** (e.g., an exponential distribution of traversal times), you would simply need to change the mean (or, in a worst case, the probability distribution), which takes half an hour at most.

The simulation workflow – validation (1/2)

Validate the model

- Are the assumptions that we made **reasonable**?
- Do the same things happen in the model as in the real system?
- If not, can we ignore the differences?

- Do this **before** you start even *thinking* of writing down code...
- ...**and** after you're done with it

- Three techniques
 - Expert intuition
 - Real system measurements
 - Theoretical results

Models need to be validated, that is, one must make sure that they correctly represent the system for which they stand in.

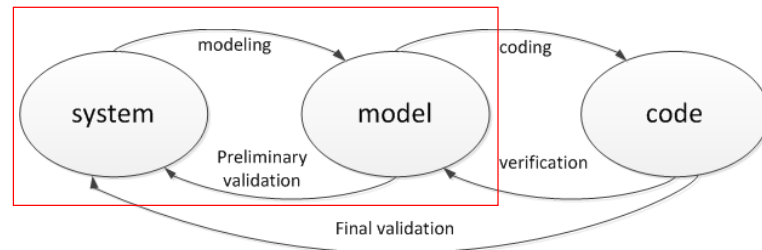
Validating a model means verifying if the assumptions that have been made to obtain that model lead to something that behaves reasonably like the real system.

You can do this using three techniques, which may or may not be available for all the models. You can use the techniques concurrently, of course.

Depending on the situation, you may not be able to do validation (i.e., when you are simulating a completely new system, that does not exist yet, for which no theoretical results are available and that no one is an expert of).

The simulation workflow – validation (2/2)

- *Preliminary validation*
 - Weak: check the model against the system
- *Final validation*
 - Stronger: check the (verified) code against the system



The initial validation is weaker: a model is a **document**, so it's hard to compare against a system. When you have the **code** (which has to be verified first), you can make the **final, a-posteriori** validation.

The simulation workflow – factors

Define the factors

- i.e., the degrees of freedom of the system model, which might affect the performance
- Factors change depending on the accuracy of the system model
- Some factors can actually be modified in the real life, while some others cannot
- A set of factor levels makes up a consistent part of a **scenario** for your simulation

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

102

One key aspect of modeling is **deciding factors**. What are the tuning knobs for my system? How much should I be able to turn them?

In the case of our 5G network, some of the factors I can define are:

- The **application traffic model**. This is a **qualitative** (i.e., not quantitative) factor.
- The **number of base stations or users** on which I want to test the VoIP performance, which is instead a **quantitative** factor
- The **queue size** at the base station side, which is quantitative
- The **interference mitigation algorithm** (seeing as I want to compare some of them using simulation), which is not quantitative again.

Some of the above can only be considered as factors in a **simulated environment**. For instance, the number of base stations in a 5G network cannot be changed in real life, since it depends on the deployment made by the network operator, which is outside my control. I can change it in a simulation environment.

When you run a simulation, the set of factor levels composes part of a **simulation scenario**.

Example, interference coordination algorithm as a factor:

- Scenario 1: algorithm A, N users
- Scenario 2: algorithm B, N users

Adding one more factor, routing delay:

- Scenario 3: algorithm A, M users
- Scenario 4: algorithm B, M users

Knowing factors gives you additional information about what pieces of the system you need to model. For example, if the interference coordination algorithm is a factor, then you cannot abstract it away from your model, and probably you cannot abstract away the computation of the received power at the user side.

The simulation workflow – tools (1/2)

Select the simulation tool

- Alternatives
 - Technology-specific existing simulator (e.g., OMNeT++ with INET, ns-2, ns-3, OPNET, GloMoSim, QualNet...)
 - Customize an existing simulator
 - Write your own simulator
 - Using simulation-oriented frameworks (Simulink, Arena, etc.)
 - Using simulation-oriented languages (Simula, etc.)
 - Using general-purpose programming languages (C++, Java, etc.)

Once you have a clear idea of what you are going to do, how to prove it, varying what, just **pause and reflect before starting to write the code.**

There are several options:

- Ready-made simulators for the type of system that you want to simulate (e.g., network simulators)
- Simulators that already include **many elements** of the model that you want to simulate, but need be **enhanced** to incorporate some of the functionalities that you require
 - Typical example: I invent a new interference coordination algorithm for 5G networks. My life will probably be considerably easier if I select a simulator that already has all the models that I need (a 5G base station model, a link model, traffic generators, a state-of-the-art interference coordination algorithm), and limit myself to **writing down the code for my algorithm only.**
- Simulation-oriented programming languages provide users with simulation-oriented data structures (e.g., event queues) and functionalities
- There are simulation frameworks where you build up a simulator by connecting blocks (as in a block diagram), e.g., Simulink, Arena, etc.
- General-purpose languages, e.g. C++ or Java. These should be object-oriented, for fairly obvious reasons

The simulation workflow – tools (2/2)

Select the simulation tool

- Straightforward factors affecting the choice are:
 - cost of the software
 - expertise with a specific simulator (or class thereof)
 - simulator capabilities and/or amount of expected modifications needed to run simulations that fulfill our objectives
 - analysis tools available for (or integrated with) the simulator

Even selecting a simulation tool is not an easy task. The choice should be made based on engineering principles and common sense.

The last item should not be neglected. A large portion of the time involved in a simulation analysis is spent in **analyzing the output data**.

Some simulators offer a varying degree of support to the **latter steps of the simulation workflow**, meaning that they can automate some of the latter steps (those that we will see at the end). If they do offer some support, this is going to save you a lot of time later on. If they don't, you should write that part of code yourself (or, god forbid, do the thing *manually*)

If you need to simulate (say) a **manufacturing plant**, it is unlikely that you will find a model in an existing simulation framework (say, OMNeT++). However, if you **do** use that framework, you enjoy:

- Event queue routines
- Good RNGs
- Analysis and workflow automation tools
- (what's more important) a **programming paradigm**: the way you build objects and have them communicate follows a precise **rigid paradigm**. This is heaven, especially if the project is on a large scale and consists in the contributions of several hands.

If you don't go for a simulation framework, you have to do all the above on your own. It's time consuming, error-prone, and will probably lead you to a dead end.

The simulation workflow - implementation

Implement or modify parts of the simulator

- According to our needs and depending on simulator capabilities
- Any coding should follow the best practices of software engineering and programming
 - Extensive **debugging** should be performed
 - Anti-bugging techniques
 - Do all probabilities sum to 1?
 - Is $\#received = \#sent + \#dropped + \#enqueued$?
 - Exceptions: the more, the better
 - Extensive, structured **testing** is also required

"If debugging is the process of removing software bugs, then programming must be the process of putting them in"

E. Dijkstra (?)

Coding must be state-of-the art, of course. Even more so in a simulator.

This is because, as we will see in a minute, there is a lot more to ask from a simulator than just being bug-free.

If using a simulation framework (e.g., OMNeT++, ns-3), take a look at its manual: they may have best practices to be followed to properly code stuff in those environments, which help you do things better (and faster)

Anti-bugging techniques: add specific code to the software that checks specific conditions. If not correct, raise an exception. It may seem counterintuitive, but sometimes in simulation you end up being happy when your simulation raised an exception... because it helped you find a (logical?) bug that would have altered your results

The simulation workflow – verification (1/8)

Verify the implemented simulation model

- Does the **code** correctly implement the **model** (which, in turn, has been **validated** to correctly represent the system)?
 - It is the hardest (and longest, and most boring) part
 - Debugging a standard program is simpler. When it does something unexpected, there is a bug
 - Here, you **don't know** what is expected and what is not
- It is **essential**. Never **ever** start getting results out of an unverified simulator

Most people believe that the code is ready when it **stops crashing**. This is obviously false.

-**Verification**: the code correctly implements the **model**. There are no “logical bugs”, i.e. the model and the code yield the same result in any case

-**Validation** (of the model, through the code): the model is a correct abstraction of the real thing. If you have the real thing, you should obtain the same results in the real thing and as an output from the simulator.

However, you should always do a proper verification. Take into account that verification **absorbs a huge amount of time**.

Never **ever** start running simulations before terminating this part. You definitely **will find some “logical bugs”**, and you will be forced to throw away everything that you have done so far.

The simulation workflow – verification (2/8)

Verify the implemented simulation model

- Run **test** simulations and **verify** the correctness of the results
 - Use a trivial simplified case at first
 - Test the behavior with deterministic models
 - So that you can compute results manually
 - Introduce complexity one step at a time
 - Re-create scenarios used in other studies (simulative, analytical or measurement-based), and check if the same results are obtained
 - Have other scientists peer-review your results

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

107

Part of the verification consists in running **simple scenarios**, of growing complexity, whose results can be **verified manually**.

If I have to test the new congestion control algorithm for TCP, I can start with a scenario with:

- Two nodes
- One link (with a null error probability)
- Just one application

In this scenario, I can do the computations **manually**. I can therefore verify the correctness of what the simulator does in that scenario. As the confidence in the correctness of the data increases, I can complicate the scenario at will

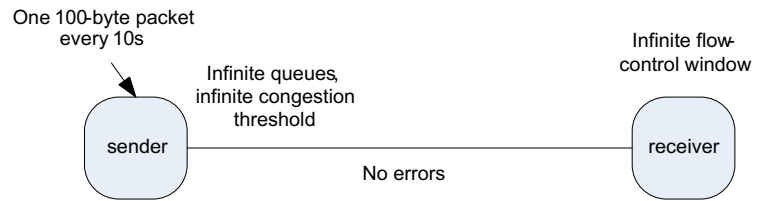
- Add more hops
- Add disturbance traffic
- Add an error probability on the link

Possibly, not all at the same time (otherwise I wouldn't know where to look for problems if any do occur).

Sometimes verification can be done by comparing **your simulator results** with something else (e.g., some other simulator's, or the real system, if available).

The simulation workflow – verification (3/8)

- Example



- Add more hops
- Add disturbance traffic
- Change the flow-control window at the receiver
- Change the congestion window at the sender
- Add randomness
 - Errors on the link
 - Random packet sizes and arrival times

The simulation workflow – verification (4/8)

- Consider using tools for incremental verification (e.g., fingerprints)

Example: modify one portion of the simulator, then verify that the other parts of it (unrelated to the first portion) behave in exactly the same manner.

The simulation workflow – verification (5/8)

Other **practical** suggestions:



- Structured walk-through (aka *rubber-duck debugging*)
 - Explain your code to someone else (possibly more than one person), line by line
 - Only step to the next line if everyone agrees on what has been done so far
 - If they can't understand what you're doing, there's probably a flaw in the code
- Add and use **event traces** to see what your code actually does
 - If an event trace has enough details, you can spot bugs by “simply” looking at it
 - Or, use scripts to spot particular conditions

Event traces: insert prints in your code to track the evolution of the model

Of course, you can also take advantage of a debugger.

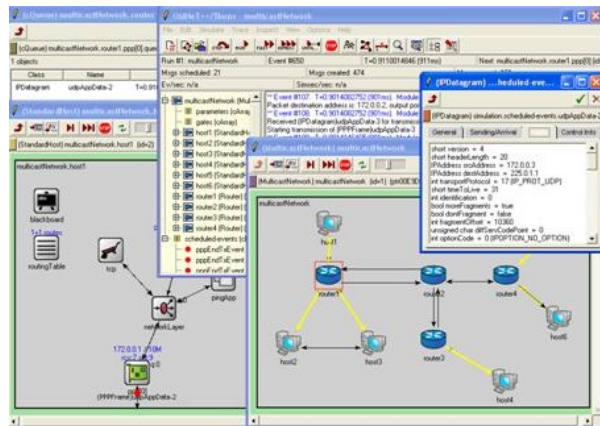
The simulation workflow – verification (6/8)

- The implementation of the model may be correct, but there may be bugs in statistics computation

Example: bugs in statistics computation.
Compute duration of successive operations using timestamps.

The simulation workflow – verification (7/8)

- Use **animations** and **graphs** whenever possible
 - Some simulation packages do this for free
 - Odd behaviors easier to spot
 - Increases saleability



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

112

The simulation workflow – verification (8/8)

- Consistency test
 - The system should react to one source sending at 100kbps much like it would to two sources at 50kbps each
- Degeneracy test
 - The system should work with zero error rate, zero propagation time, infinite windows/queues, zero users, etc.
- Continuity test
 - If $input_j \rightarrow output_j$, then one would expect that changing slightly the input does not change the output wildly
 - Check for non-monotonicity and mysterious or odd behaviors (e.g., outliers)

For instance, if you change the link error probability by 1%, then you would not expect that the throughput would change dramatically. Check that it does not.

Most of the times you can find good continuity/consistency tests for your simulator by leveraging your experience of the system being simulated (or asking someone to confirm this)

The simulation workflow - calibration

Calibrate the simulator

- Tune the simulation parameters (**factors vs parameters**)
 - So that scenarios are realistic
 - Consistently with the objectives
- In a 5G network, e.g., the # of base stations should be set according to our target scenario:
 - With one base station there will not be interference
 - Two base stations might be not enough to assess the impact of inter-cell interference
- Configuration parameters common to all dynamic simulations:
 - Simulation duration
 - Warm-up period duration (both discussed later)

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

114

In a simulator there are tuning knobs, or – better yet – **configuration variables** which should be given a value. Some of these are **factors** of the simulation, i.e. those that you want to change in the analysis. Others are simply **configuration parameters**, which you will keep constant. The set of all these represents the **simulation scenario**.

Scenarios must be **carefully engineered**. They should mirror operating conditions which are **real-life-like**, and **apt to verify your claim**, i.e., consistent with your objectives.

Regarding **realism**:

-It makes no sense to test Internet connections with 60 hops. Paths are never that long (30 hops get you to New Zealand). The number of hops in the Internet is bound to **decrease** over time (since connectivity is on the rise). The fact that my congestion control algorithm works well/fails on paths with more than 50 hops is **totally irrelevant** to its marketability.

-It makes no sense to test TCP using applications that **send 1 byte/day**. No applications do this, and those that do never congest links.

Regarding consistency with your objectives:

-It is still a good laugh (among scientists) to recall that (see Camp, 2004) ad hoc network routing protocols have been tested for years on a network scenario where the average number of hops between 2 endpoints was 1.2. This means that more than 80% of the times the sender and receiver are directly connected, hence no routing takes place.

There are, in particular, two parameters that have to be considered **very carefully**, since they make up a large part of the (in)credibility of your simulation study

-The simulation duration

-The warm-up time (think about computing average speed of a 100m runner)

The simulation workflow – experiment design

Design the simulation experiments

- Consists of two steps:
 - select the range of values of interest (**levels**) of each factor
 - remove factors whose impact on the defined performance indices is negligible
- Difficult in general, usually done *empirically*
- There are known techniques, like $2^k r!$ analysis

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

115

You need to decide the **range of values** for each factor.

Note that (by the basic principle of counting), if you have two factors, with n_1 and n_2 values each, you would need $n_1 \cdot n_2$ experiments to test all the possible combinations. This is clearly impossible when the number of factors and values grows beyond a few units, since the number of experiments grows **exponentially**. On the other hand, you should be aware that factors may **interplay in strange ways**, so that very peculiar outcomes may be produced when two or more factors have a certain relationship (which you don't know in advance, of course).

For instance, I might vary the factors as follows:

- # of hops, from 2 to 30
- Link bandwidth, from 1Mbps to 1Gbps (possibly logarithmically)
- Bit error rate, from 10^{-10} to 10^{-3} (again, possibly logarithmically)
- Buffer space on the nodes, say from 4kbyte to 1Mbyte

I can't state in advance whether the bandwidth plays a different role in conjunction with large/small buffers. This puts a simulator **user** in a very uncomfortable position. There are techniques to reduce the number of factors. One is **$2^k r!$ analysis**, which tells you at a reasonable cost which factors have the highest impact on the performance.

Other than this, you can only use your expertise on the system.

The simulation workflow – running simulations (1/2)

Run simulations

- This step requires variable time, mostly depending on the complexity of the simulations and the accuracy required
 - The hardware used to host the simulations has a role
 - The type/number of recorded statistics has a role
- You may want to disable anti-bugging and trace logging features when running real simulations
 - At this time **all bugs** must have been corrected
- For large simulation campaigns, it can be useful to run simulations in parallel over multiple machines:
 - Parallel Discrete-Event Simulation (PDES)
 - Multiple Replications In Parallel (MRIP)

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

116

Running simulations **takes time** (and a lot of it, in many cases). For this reason, **think before running a simulation**.

- Is this the **one scenario** that I want to test?
- Are the parameters **set to the right values**? Are they **mutually consistent**?

It happens all too often that some parameters have been set to one value or another without giving it much thought, and this choice invalidates days and days of work.

Simulation can be hardware-hungry. Normally **memory** plays a major role (because data structures, especially packets, use up a lot of memory), and disk swaps should be avoided if possible. Multicore processors can be used to distribute the load.

Now, if you want to speed up your simulation, you have two alternatives:

- **Go parallel**: this is, however, a **design choice**. If you have designed your simulator to be parallel, you can distribute it on several cores/machines and harvest some (sublinear) speedup
- Distribute **several scenarios** (or, better yet, **replicas** of a scenario) among several PCs (MRIP) (or several cores of your processor).

Of course, you should not be doing this **manually**, but you need a **simulation automation software** that does this for you.

The simulation workflow – running simulations (2/2)

Run simulations

- Parallel and distributed simulation
 - Do parallelization on a single run
 - A) several components can be distributed among processors, e.g., one for the event queue, one for the event routine, etc.
 - B) several **parts** of the model are autonomous (e.g., different cells of a cellular network), and their execution can be allocated to different processors
 - Each part carries on autonomously, **message passing** coordinates the parts
 - Deadlocks are possible
 - Rollback in time may be required
 - Do parallel runs
 - Treat each run as independent program (as it actually is)
 - Launch multiple processes simultaneously

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

117

P&D simulation is an active research field, which we will not deal with in the course of this lecture.

Of course, designing a simulator to be parallel and/or distributed is an entirely different task. The main problem is to maintain a consistent, shared notion of **time**.

Module X might generate event x at time t, which requires module Y to do something at time t.

Meanwhile, module Y could be already working at time t+D, and generating events of its own. If it gets an event for time t (i.e., earlier in time), then it needs to check whether everything that it has done in [t, t+D] is still meaningful. If it is not, it should roll back in time and take a different path.

When you play with synchronization, you always run the risk of incurring deadlocks.

The important thing to know is that a simulator is **designed from the start** to be either sequential or parallel.

With **parallel simulators**, the speedup due to parallelism can be large to huge, especially if the modules are **semi-autonomous** and **computation-intensive**: they do a lot of processing and little interaction.

If, instead (as happens with networks) the modules do **very little processing**, and have **a lot of interactions**, then it is preferable to go sequential. Going parallel buys you no speedup at all.

The simulation workflow – data collection

Collect and organize output data

- Very important for large simulation campaigns
- **Record the exact conditions** under which the data have been obtained
 - With poor storage organization, data become unrecoverable
 - The complexity of distinguishing which data belonged to which simulations becomes comparable to that of running the simulations again

When you simulate **in the large**, before you start producing results, you need to tackle the problem of **storing the results**.

It is quite common to produce hundreds of MB of data, and a **file system is not up to the task**. More specifically, a file system does not offer you any support to **associate a scenario file** to an **output file** that contains your metrics. If you rely on the file system alone, you have to do the association **manually**, which is **utterly error-prone**:

- You need to create hundreds of folders
- Replicate the right scenario file in each of the folders
- Enforce *manually* a hierarchy on the factors (since the file system *is* structured hierarchically)

The correct solution is to **use a relational database** to store **both the scenarios and the results**.

- Or use a simulation environment that helps you with this step.

The simulation workflow – data analysis

Analyze results

- In this step the performance indices have to be extracted from the raw data and visualized, in combination with the possible values of the relevant factors
- 2D and 3D graphs, tables, animations, etc. can be used to this purpose
- Follow usual guidelines on data and graph presentation
- Your task is to generate **insight**, not **numbers**.

The raw results need to be analyzed to produce something usable: **graphs, tables, animations**, etc. It is up to you to locate the result that best shows **your research claim**. For instance, when you compare your invention against something existing, a good practice is to test your invention in the **most unfavorable conditions**. If it outperforms the competition in that scenario, it will do the same in all others.

You should rely on the **quality** of the information you produce, not on the **quantity**. When doing a presentation, no one will pay attention to more than 3-4 graphs.

It is not necessary to show **how much work** you have done, e.g. showing all the graphs even when they are meaningless *just because you took the pain to draw them*.

Why do simulation studies sometimes fail?

- No SMART (Specific, Measurable, Achievable, Relevant and Time-bound) goals
 - Inadequate time/cost estimate
 - Wrong mix of required skills
 - Inadequate level of detail
- Lack of validation and verification
 - Incorrect assumptions
 - Mysterious results go unexplained
- Unsound analysis of output data
 - Bad RNGs or seeding thereof
 - Wrong assumption of IID-ness of outputs
 - Mistaking transient for steady state
 - Wrong “mixing” of multiple outputs

By “fail” we mean that we get **meaningless** results (or no result at all).

Output data analysis

“The average human has one breast and one testicle”

“Statistics are used much like a drunk uses a lamppost: for support, not illumination”

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

Most people think that a simulation project involves modeling the system and programming a computer software. Then, running a single simulation is enough to obtain credible results. This is obviously false.

“The average human has one breast and one testicle” – Des McHale

“Statistics are used much like a drunk uses a lamppost: for support, not illumination” – Andrew Lang

More aphorisms:

“Facts are stubborn, but statistics are more pliable.” – Mark Twain

“There are three kinds of lies: lies, damned lies, and statistics.” – Mark Twain

Analyzing output data

- An output measure from a simulation run can be
 - a **random variable**
 - the throughput of TCP over a simulation run
 - a **stochastic process**
 - the delay of subsequent packets in a simulation run,
 - $D_1, D_2, \dots, D_n$
- Randomness at the **output** follows from randomness at the **input**
 - Repeat the same experiment with different initial seeds -> different results

Simulation

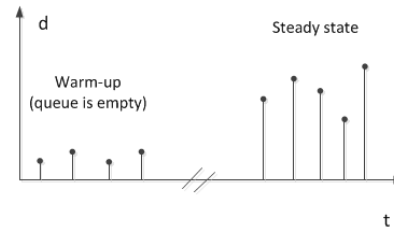
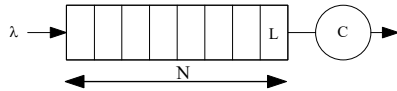
Ing. Giovanni Nardini - University of Pisa - All rights reserved

What you obtain from a simulation is – almost always – random: if you repeat the simulation with different initial seeds you obtain different results.

The output can be a single variable (first case), or a **process** (second case, which includes the first one). In the second case, samples cannot even be assumed to be IID, hence most of the formulas you know from the theory cannot be applied directly.

The solution is to repeat the simulation N times, which produces N realizations of the stochastic process: samples obtained from different runs are independent.

Steady-state analysis



- We are interested in what a system does
 - In the **long run**
 - Under **normal operating conditions**
- Experimental evidence shows that simulated models often reach a **steady state** (i.e., normal operating conditions) after some **transient** (warm-up period)
- We need means for
 - Estimating the length of the transient
 - Figuring out how long the “long run” should really be

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

123

If you start with an empty queue+server, the first few packets will have a **very low delay** and will not be dropped, which may not be representative of what a packet can expect when arriving after, say, 1000s from the start. If you derive results using all the samples from a simulation you might end up having results that are not representative of the system you wanted to study.

Thus, we are interested in the **steady-state behavior** of a system, i.e. the behavior that it reaches once the transient phase is extinguished.

This poses the problem of **estimating the length of the warm-up phase**, because you should **avoid taking samples** before the system is in a steady-state, lest you set a **bias** on your statistics.

Note that **models** often reach a steady-state. **Systems** seldom do in practice. This is because the conditions that you set at the beginning of the simulation (and assume true for the whole duration of the simulation) do change in reality. For instance, the number of hops in a TCP connection is bound to change because routing is dynamic in the Internet. Or the number of users served by a cellular base station changes because of user mobility. In the **model**, you can fix such parameters, whereas in **reality** they do change.

Still, not all models have a steady state. For example, a routing protocol is designed to exchange messages among network nodes to disseminate routes at the initialization of the network, or when a failure or reconfiguration is needed. After routing tables become stable, the routing protocol does not exchange messages anymore (except for some keepalive messages every now and then). If you want to evaluate the time needed to reach routing table stability you are actually observing a transient of the network by definition.

Estimating the length of the warm-up period

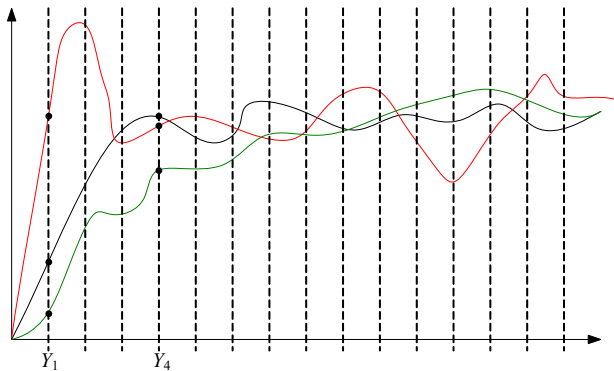
- Common approach: cut the first 10%
 - Makes no sense at all
 - Why 10%? Why not 20% or 1%?
- Correct approach:
 - Make n **independent replications** of the same scenario
 - Collect a (relatively large) **sample** of the quantities that you want to estimate
 - Make statistical inferences on those quantities

You can do this, as usual

- Using some **common sense**: you may know something about the **time constants of the system**. If you know that all your traffic sources activate in the first 10 seconds, and that they send 100 packets/second on average, it is reasonable to think that your system will be in a steady state starting from 12-13.
- Using **statistical tests**: you can take a moving average of some quantities (over a time window), and see when they stabilize around a constant value.

Estimating the length of the warm-up period

- Delay of single packets over time for $n=3$ **independent** runs
 - Y_1 and Y_4 are random variables
 - A sample of three observations is available for each RV



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

125

If you use **different seeds for the random generators**, then you make **independent replicas** of the same scenario, and this is the statistically sound way to do things.

If you consider the delay of the j -th packet in each replica (the same across all replicas), this is a random variable, whose distribution you can **estimate** using statistical techniques.

If, starting from a certain packet j , the distribution (or, at least, the **mean**) stabilizes to a constant value (or around a constant value), then we can say that the system is in a steady state.

You should find this value j , and discard everything that happens earlier. Statistical analysis made on remaining samples does not depend on the initial conditions, i.e., on RNG seeds and initial system status (such as active sources, number of packets in a queue, etc.).

Estimating the length of the warm up

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

126

How to proceed:

- Make N replicas with different RNG seeds (a reasonable number of observations is around 10 - three is way too small)
- Compute the mean across replicas for each sample $Y_j \rightarrow E[Y_j]$
- Plot the $E[Y_j]$ for $j = 1, \dots, m$. In order to smooth out the curve, you can use a sliding window average mechanism.
- Identify the j^* for which the $E[Y_j]$ s converge (does not mean that they must have the same value)

A quicker (less accurate) way is to plot all the replications and observe when they all converge to a similar value. Also in this case a sliding window average can be used to smooth out the realizations.

If oscillations still make it hard to estimate the warmup, you can observe when the sample mean of each realization converges (note that this gives you the time when the **mean** converges, not the **samples**).

Steady-state analysis – Simulation duration

- How long should a simulation be?
 - Long enough to collect a statistically meaningful number of events (in the steady state)
- Example:
 - Suppose you simulate a link with a given error model, and you want to estimate the packet loss probability
 - From some a-priori knowledge, you expect the packet loss probability to be $\sim O(10^{-4})$
 - You need to simulate the transmission of more than 10^4 packets in order to obtain statistically meaningful results
 - Possibly 100 times as much as that

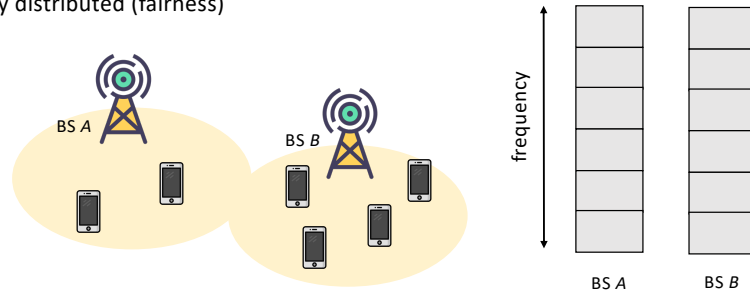
This is a bit simplistic, of course. We do know more advanced techniques (e.g., related to the confidence intervals for Bernoulli experiments).

Computation of average metrics

- Average metrics are computed considering the simulation duration
 - e.g., throughput
- Remember to discard the warm-up period!

Steady-state analysis – Case study

- Compare **interference mitigation techniques** in a **5G cellular network**
 - Measure the **throughput** of different UEs receiving VoIP traffic
 - **Claim:** when base stations coordinate, the throughput
 1. increases (efficiency)
 2. becomes more evenly distributed (fairness)



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

129

This slide recalls the claim we want to prove in the considered case study. We want to measure the **throughput**, i.e., the following expression: $t_j = [\# \text{ packets received by UE } j] / [t_1 - t_0]$.

Our claim is that the throughput:

- a) Increases in general;
- b) Becomes more evenly distributed (i.e., **fairer**).

If coordination among antennas is used.

Throughput is of course a long-term measure, hence should be measured in the steady state. It is random, as it depends on the effects of the interference in the cellular networks, and because it depends on the input traffic, which is variable itself

Steady-state analysis – Case study

- VoIP traffic
 - 1 fixed-size packet every 20ms during **talkspurts**
 - No traffic during **silences**
 - Talkspurts and silences are random, mean duration $\sim O(1s)$

- 1) How do I detect when the initial transient is over?
- 2) How long should the simulation be?

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

130

In a system where the resources are **finite** (the # of RBs to be shared among the UEs), we have a steady-state situation if the number of **active** VoIP sources (i.e., those in a talkspurt) is **statistically** constant over time. We should therefore ask ourselves when these conditions are verified. This helps us to give a value to t_0 in the above window.

In order to verify this, we run **the same number of VoIP sources without the 5G network**, and plot the number of those that are in a talkspurt at any given time.

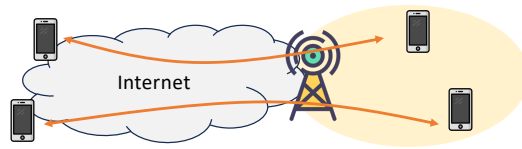
How should I simulate this network? For how long? When should I start taking measurements?

The rest of these few slides show how to determine

- a) When the initial transient is over
- b) How long the simulation should be in the steady state.

Steady-state analysis – Case study

- Understand the VoIP traffic dynamics **first**



Too many simultaneous causes of variability



Simplified scenario
Start from this!

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

131

In our case-study, we need to show how **throughput changes** due to the interference conditions, and what happens if you eliminate interference.

My point is: sources are **random**, so there will be differences in throughputs which are due to **randomness** in the activity ratio, and others that are due to **differences in interference**. I want to measure the impact of the **latter only**, and discount the effects of randomness as much as possible.

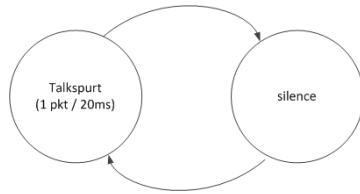
So, I need to understand **randomness first**.

In order to understand it, it is preferable to **simplify the scenario as much as possible**. Try running the same number of VoIP conversations **without** a 5G network in the middle first, and only **then** you put it back.

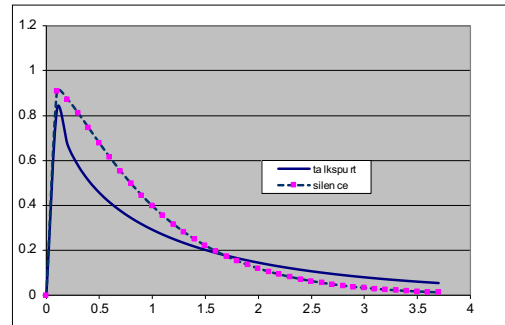
The network in the simplified scenario is **as ideal as it can be**: an overdimensioned bandwidth, no wireless effects (such as packet losses or retransmissions), no core-network delay, etc.

In the above example, the only sources of randomness are the **different activity ratios**.

Steady-state analysis – Case study



- Highly likely to find **long** talkspurts/silences
 - Median is 0.91s/0.64s
- Sources are in a talkspurt when started



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

132

Let us take a look at what traffic is transmitted in this simulation.

The distributions of the duration of the talkspurts and silences are Weibull, with the above median. Their **mean** is even higher, since distributions are skewed to the right.

By looking at the above PDFs, it is quite clear that you can encounter **long** silences and talkspurts with a non-negligible probability. By “long” I mean 4-5 seconds, or 5-10 times the median. This will have some bearing on our selection of the warm-up and simulation duration.

A typical way to set the duration of the transient and the duration of the simulation is to look at the activity ratio of the users, i.e. the percentage of users that are in a talkspurt period.

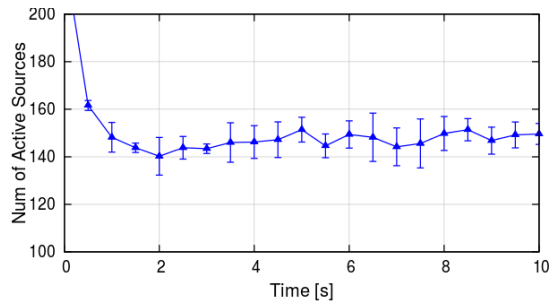
This activity ratio can vary a lot over time but eventually will converge to an average value of $N \cdot$

$$E[t_{\text{talk}}]/E[t_{\text{talk}}] + E[t_{\text{sil}}] = N \cdot 0.65.$$

So, we need to find a way to discover when the average activity ratio is statistically constant in order to get measurements from the system in a steady state.

Steady-state analysis – Case study

- Estimate the warm-up phase:
 - Start sources at time $t=0$, and measure the RV “# of active sources” over time
 - Repeat 5 times in independent conditions
 - Sample the above RV at time 1, 2, 3, 4,...
 - Compute mean and variance



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

133

In order to verify this, we run **the same number of VoIP sources without the 5G network**, and plot the number of those that are in a talkspurt at any given time.

All sources start at $t=0$ (by design of the simulation model for VoIP).

We observe the following:

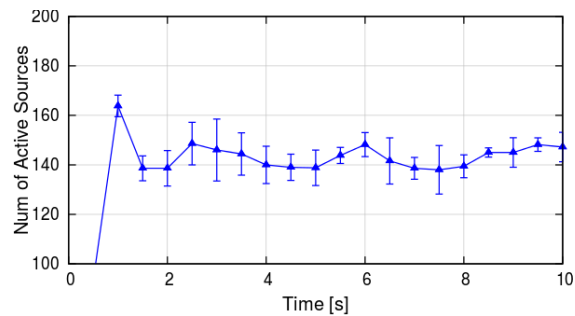
- initially, all sources are in a talkspurt (see the left part of the graph)
- After a while, the randomness of the duration of talkspurts and silences takes over, and the number of active sources evens out to a steady-state number
- That number should be equal to $N * E[ts] / (E[ts] + E[silence]) = 0.65 * N$ (otherwise something is wrong in the simulation, e.g. random number generation).

Our conclusions are that the **warm-up** period, in this case, lasts until 3-4 s, so we should set **$t_0=4$** .

Can we do anything to reduce the initial transient? Not much of a problem here, but let's assume it is.

Steady-state analysis – Case study

- A possible solution to reduce warm-up period
 - Start the sources at **random times** in $[0;1]$
 - Repeat the procedure of the previous slide



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

134

One idea: instead of starting **all the sources at time $t=0$** , we might try to **start them at random times**, e.g. in $U(0,1)$.

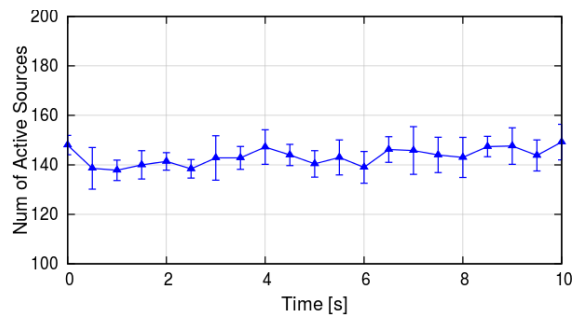
The result is that the transient does not end sooner. In fact, in this case too we see that it is not safe to start collecting measures before 2-3 s.

The idea did not have any good effect.

Steady-state analysis – Case study

- An alternative

- Start the sources in a **random state** at $t=0$ (with a probability equal to the activity ratio)
- Repeat the procedure of the previous slide



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

135

We start a source in a talkspurt state or in a silence state **at random**, with a probability $p=E[ts]/(E[ts]+E[silence])$. This way, the number of sources in a talkspurt is **(stochastically) constant** over time. The RV appears to be in a steady state right at time $t=0$.

So, depending on how you initialize your scenario, the initial transient will terminate sooner or later. It would not be a problem in any case in this simulation, since the transient is **short** in any case. But still, it pays to look at the problem carefully.

Steady-state analysis – Case study

- Estimating the **simulation duration**
- Different **throughputs** may be due to
 - Different interference/channel conditions
 - This is what we want to measure
 - Different **activity ratios** for the various sources
 - The throughput is always $\leq$ the offered load
 - Silence/talkspurt distribution may introduce a **bias**

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

136

We now have the problem of selecting **t1**. How long should the simulation be?

We are measuring the **throughput**. The latter is the **# of received packets** over a window of time, and cannot be higher than the **# of transmitted packets** in the same time window.

Since the ts and silences have **random duration**, the last quantity will be **random**. The question is: how **variable is it?**

Because if it is too variable, I will never be able to tell whether differences in **throughput** are consequences of:

- a) The differences in **interference conditions** (which is what I want to measure)
- b) The differences in **activity ratios** (which is just a nuisance I would gladly do without).

Assume that I measure the above network for 10s after removing the initial transient, and that I spot some differences in throughput.

I cannot say for certain that those differences are due to **different interference conditions**, because they may be due to the fact that **one source has much longer silences than another**, and this may affect its throughput **more than interference would do**.

There is a possible bias due to the different **activity ratios**, hence I should remove it before attempting to draw a conclusion (recall the discussion on the **scientific method**: do not believe a conclusion unless it is thoroughly proved).

How can I remove the activity ratio bias?

Steady-state analysis – Case study

- Solution 1:
 - Measure the throughput for all the users in $[t_0, t_1]$ for increasing t_1 , and stop when the distribution stops changing
 - Difficult to compare distributions
 - Throughput depends on the coordination scheme, so you may end up measuring **both** at the same time
- Solution 2:
 - Measure the **activity ratio** of VoIP sources in $[t_0, t_1]$ for increasing t_1
 - Activity ratio for a source: $[\text{total length of talkspurts}]/[t_1 - t_0]$
 - Its sample mean should converge to $E[ts]/(E[ts]+E[\text{silence}])$
 - Otherwise **do** check your RNG and your code
 - Stop when sample **variance** is small enough

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

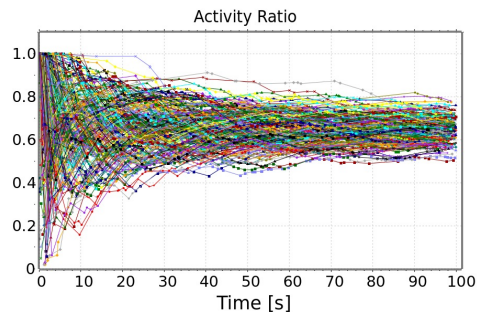
137

If I choose the first option, then I will have some difficulty understanding which effect is due to which cause. The second alternative is definitely preferable, because it does not require running the whole system - you can **disable everything** in the simulator except your traffic sources, and have them run

Now, the activity ratio in $[t_0, t_1]$ will be a **random variable**: each source will provide one sample. Samples will be independent because they are generated using different RNG instances, with “good”, well-spaced seeds. Therefore, I can compute the (sample) mean and variance of the AR of the N sources in $[t_0, t_1]$.

- The sample mean should **converge to** $E[ts]/(E[ts]+E[\text{silence}])$ as t_1 grows larger (otherwise there *is* a problem).
- What I am really interested in is its **sample variance**: if this is large, then it means that **large differences in the activity ratios among UEs should be expected**, hence large difference in throughputs will ensue from them, whatever coordination scheme I adopt. If, instead, the sample variance is small, then the differences in AR will be negligible.

Steady-state analysis – Case study



Time (s)	Mean	std
10	0.748	1.305
20	0.645	0.130
30	0.650	0.105
40	0.649	0.089
50	0.651	0.077
60	0.648	0.071
70	0.647	0.063
80	0.651	0.063
90	0.648	0.063

- If you want to get an **unbiased** measure of the throughput, then sample it in [t0,100]
 - The warm-up has extinguished
 - Activity ratio biases are small enough
 - Remaining differences are due to interference/channel and/or resource contention

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

138

Each line in the figure reports the activity ratio of **one source** as a function of the simulation time. We can clearly see that, for smaller simulation times, there is a wide spread, and that the spread gets smaller as the time window increases.

The table reports the **(sample) mean and standard deviation** for the activity ratios as a function of the simulation time. We see that the **sample mean converges quite fast** (it is stable already at t=20). In fact, even though all sources start in a talkspurt (hence the sample mean of the AR is initially overestimated), after a small while there is enough randomness to even out all the differences.

However, the **standard deviation** is still quite large at t=20. It is:

- At t=20, equal to 0.13
- At t=90, equal to 0.063 (i.e., one order of magnitude below the mean)

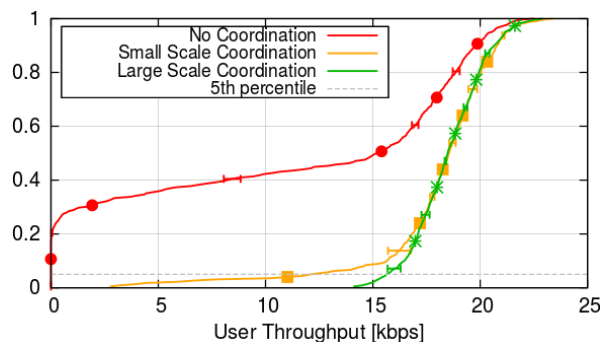
Assume that the distribution of the activity ratios among all VoIP sources at any one time is **Normal**. This means that, if you stop your simulation at t=20, your normal distribution will be $\sim N(0.645, 0.13^2)$, thus you will have 95% of your activity ratios in $[\mu - 2 \cdot \sigma, \mu + 2 \cdot \sigma] = [0.385; 0.905]$. This means that large differences in throughputs may only be due to similarly large differences in the activity ratios. The ratio between the 97.5th and 2.5th percentiles of the activity ratios (i.e., the extremes of the above interval) is 2.3.

If, instead, you simulate for 90 seconds, you will find that 95% of the activity ratios are in $[\mu - 2 \cdot \sigma, \mu + 2 \cdot \sigma] = [0.519; 0.771]$. The ratio between the 97.5th and 2.5th percentile of the activity ratios is 1.5, which is a lot smaller than before.

Moreover, you notice that **the std stops decreasing** after a while. This means that probably it will not get any smaller if you simulate that system for a longer time. After 100s, then, it is probably as small as it will ever get, and you will have to live with the remaining throughput unfairness.

Steady-state analysis – Case study

- Plot the **empirical CDFs** of the throughput using different coordination schemes
 - The benefits of coordination are correctly illustrated



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

139

We wanted to show that coordination:

- Increases the throughput in general
- Yields a **fairer** distribution of throughput among UEs.

What is in this graph? Is this the most effective way to prove these claims? What are the alternatives?

What is a **benchmark** to compare our solution against?

-**Pessimistic baseline**: no coordination

-**Optimistic baseline**: each station gets exactly the same throughput, equal to the voice activity ratio times the packet injection rate during talkspurts (i.e., packet size/20ms bps)

The optimistic baseline does not take into account the fact that **activity ratio biases** are still present in the graph, if we simulate for 100s.

Therefore, we may want to compare our graph against a **more realistic optimistic baseline**, which is the distribution that we obtain **without the 5G system at all**, which takes into account the activity ratio bias as well.

Back-of-the-envelope computation: if you try to figure out the **97.5th** and **2.5th** percentiles of the green curve, you end up with (roughly) 22 and 15. Their ratio is ~ 1.47 , which is **equal to the ratio of the same percentiles in the AR** (see the previous slide). This suggests that, if you use the “green” solution, then your throughputs will be distributed **pretty much the same as the activity ratios**: in other words, you are doing the best you can, since you are **not adding any more unfairness**.

Steady-state analysis – Take-home lesson

- **Always** test your (and everyone else's) models before using them
- **Test scenarios**
 - Most errors are made when running scenarios that would immediately appear incorrect to anyone who cares to look
- Identifying the initial transient and setting the length of the simulation **takes time and insight**
 - *Do not* overlook these aspects

Again, performance evaluation is **an art**. Art takes **patience and craft**. There is no easy, mechanical way to do things

Analyzing output data (rep.)

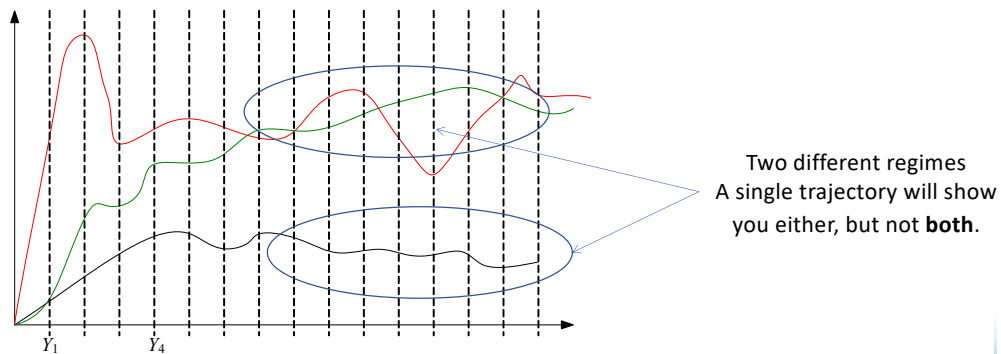
- An output measure from a simulation run can be
 - a **random variable**
 - the throughput of TCP over a simulation run
 - a **stochastic process**
 - the delay of subsequent packets in a simulation run,
 - $D_1, D_2, \dots, D_n$
- Randomness at the **output** follows from randomness at the **input**
 - Repeat the same experiment with different initial seeds -> different results

What you obtain from a simulation is – almost always – random.

It can be a single variable (first case), or a **process** (second case, which includes the first one).

Output measure analysis - representativeness

- Never **ever** attempt to make inferences on the statistical properties of a **stochastic** system by looking at a single **trajectory** of its output
- A single **trajectory** may not represent the behavior of the system



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

142

You obviously **do not know** the distribution of D (if you did, you wouldn't be simulating your system).
From the output of a **single run of a simulation** I cannot make inferences on any statistical property of the quantity that I am measuring.

In order to produce insight on a system like this, you need to acknowledge the fact that it may work in either of two regimes, each one of which has a very different steady state, and that each regime may occur with a given probability.

You are never going to see this unless you repeat your simulation so as to observe both behaviors.

Output measure analysis - IIDness

- Points in a trajectory are **never** independent
 - The delay of packet k is strongly (positively) correlated to the delay of packet $k-1$
- High-resolution measurements are **seldom, if ever**, independent
- Even though we remove the initial bias, still the dynamics of the simulated system depend on the **initial conditions**, e.g. the RNG seeds
- To obtain statistically sound output, we need to obtain **IID observations** of the phenomenon that we are investigating

Normally, the higher the resolution of a measurement (in space or time), the more likely it is that subsequent observations are **positively correlated**, instead of independent. This is because physical quantities do not change abruptly in space or time.

How to obtain IID samples

- **Independent replications**

- Run N replicas of the same simulation scenario with independent initial conditions
- Need to discard N initial transients

- **Batch means**

- Perform one “long” simulation, and slice it into N intervals
- Assume each interval is independent of the others (which is seldom true)
- Only discard one initial transient

Independent initial conditions means “using different seeds for the RNGs”

Displaying results

log100-3	log100-4	log100-5	log100-6	log100-7	log150-0	log150-1	log150-2	log150-3	log150-4	log150-5	log150-6	log150-7	log200-0	log200-1	log200-2	log200-3
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0.006726	0.006621	0.006789	0.006858	0.006917	0.005853	0.005475	0.005957	0.005944	0.005854	0.005969	0.006044	0.006023	0.00539	0.005316	0.005326	0.00554
0.006923	0.006764	0.006871	0.006972	0.007003	0.005998	0.005697	0.006085	0.006023	0.005896	0.006077	0.006088	0.006093	0.005563	0.005409	0.005465	0.00558
0.007	0.006866	0.006931	0.007029	0.007065	0.006032	0.005758	0.006118	0.006067	0.005927	0.006102	0.006111	0.006129	0.005612	0.005496	0.005558	0.005619
0.007068	0.0069	0.006977	0.007061	0.00711	0.006128	0.005875	0.006147	0.00613	0.005963	0.006124	0.006142	0.006143	0.005632	0.005496	0.005593	0.005659
0.00711	0.006929	0.007026	0.00709	0.007151	0.006148	0.005903	0.006173	0.006176	0.005987	0.006132	0.006157	0.006168	0.005705	0.005531	0.005611	0.005667
0.007161	0.006951	0.007049	0.007145	0.007181	0.006185	0.005929	0.006205	0.006187	0.00601	0.006137	0.006174	0.006183	0.005735	0.005547	0.005632	0.005697
0.007204	0.006966	0.007067	0.007153	0.007202	0.006218	0.005954	0.006215	0.006215	0.006035	0.006144	0.006188	0.006202	0.005766	0.005561	0.005645	0.005701
0.007246	0.006988	0.007097	0.007184	0.007208	0.006241	0.005989	0.006236	0.006215	0.006047	0.00616	0.006203	0.006209	0.00578	0.005594	0.005661	0.005711
0.007265	0.007023	0.007109	0.007199	0.007215	0.006272	0.00601	0.006248	0.006223	0.006056	0.006165	0.006222	0.006225	0.005797	0.005602	0.005671	0.005719
0.007291	0.007047	0.007146	0.007211	0.00722	0.006286	0.006036	0.006265	0.006229	0.006064	0.00618	0.006229	0.006238	0.005813	0.005613	0.005695	0.005731
0.007317	0.007061	0.007171	0.007236	0.007236	0.006316	0.006061	0.006272	0.006242	0.006078	0.006184	0.006244	0.006244	0.005847	0.005646	0.005716	0.005745
0.007333	0.007077	0.007184	0.007245	0.007247	0.006347	0.006078	0.006278	0.006252	0.006083	0.006194	0.006248	0.006255	0.005856	0.005654	0.005725	0.005753
0.007356	0.007095	0.007191	0.007258	0.007264	0.006357	0.006094	0.006283	0.006273	0.006096	0.006206	0.006256	0.006263	0.005874	0.00567	0.005732	0.00577
0.007372	0.00712	0.0072	0.007266	0.007277	0.006364	0.006101	0.006295	0.006279	0.006105	0.006216	0.006264	0.00627	0.005883	0.005682	0.00574	0.005781
0.007386	0.007142	0.007211	0.007287	0.007288	0.006379	0.006114	0.00631	0.006286	0.006111	0.00622	0.006273	0.006277	0.00592	0.005689	0.005752	0.005798
0.007411	0.007158	0.007223	0.007307	0.007302	0.006418	0.006138	0.006323	0.006298	0.006118	0.006226	0.00628	0.006285	0.005933	0.0057	0.005759	0.005802
0.007428	0.007175	0.007226	0.007316	0.007305	0.006436	0.006155	0.006335	0.006308	0.006125	0.006232	0.006291	0.006297	0.005938	0.005702	0.005761	0.005807
0.007438	0.007191	0.007246	0.00732	0.007321	0.006444	0.006177	0.006338	0.006322	0.006133	0.00624	0.006294	0.006306	0.005944	0.005721	0.005767	0.005813
0.007462	0.007222	0.007251	0.007335	0.007327	0.006462	0.006184	0.006345	0.006331	0.006136	0.006247	0.006299	0.00631	0.005951	0.005727	0.005773	0.005819
0.007473	0.007237	0.007268	0.007347	0.007339	0.006475	0.006199	0.00635	0.006339	0.006142	0.006253	0.00631	0.006316	0.005961	0.005735	0.005776	0.005826
0.007479	0.007237	0.007268	0.007364	0.007354	0.006481	0.006209	0.00636	0.006344	0.006154	0.006261	0.006314	0.006321	0.00598	0.00574	0.005788	0.005829
0.007499	0.007248	0.007276	0.007368	0.00736	0.006496	0.006218	0.006364	0.006356	0.006171	0.006265	0.006317	0.006331	0.005995	0.005748	0.005799	0.005837
0.007513	0.007258	0.007289	0.007383	0.007366	0.006506	0.006227	0.006372	0.006368	0.006182	0.006275	0.006324	0.00634	0.006008	0.005756	0.005802	0.005839
0.007517	0.007268	0.007296	0.00739	0.007371	0.006514	0.006236	0.006375	0.00638	0.006185	0.00628	0.006329	0.006347	0.006017	0.005763	0.005807	0.005848

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

145

At the end of the day, you will end up with a lot of numbers. You will be required to use said numbers to explain something.

Generally, using them as they are is a **bad** idea

Plots: type

- Select the proper chart type
- Most common:
 - Bar charts
 - Line charts
 - Pie charts
 - Scatter plots

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

146

When selecting the right type of visualization for your data, think about your variables (string/categorical and numeric), the volume of data, and the question you are attempting to answer through the visualization. Also, think about who will be viewing the data and how you can best optimize the data narrative through design.

Bar charts: the simplest bar charts – those that illustrate one string and one numeric variable – are easy for us to visually read because they use alignment and length. Additionally, bar charts are good for showing exact values.

Line charts: excellent way to show change over time. Use line graphs when you have one time variable and one numeric variable. Bar charts can also demonstrate time, but they fail to show the continuity that a line graph can provide.

Pie charts: good to show a part-to-whole relationship (when the total amount is one of your variables and you'd like to show the subdivision of variables).

Scatter plots: useful for showing precise, data dense visualizations, correlations, and clusters between two numeric variables.

Plots: number of curves

- Use a reasonable number of curves in the same plot

Make sure your chart is readable

Plots: coloring + markers + line type

- Select a proper color coding

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

148

Use color meaningfully, aka use different colors only when they correspond to differences of meaning in the data

- categorical (different colors for different categories)
- diverging (from red to orange, green, blue)
- sequential (different grading of the same color to show quantitative differences, e.g. from light blue to dark blue)

Consider the format of your visualization, e.g. will it be displayed in a color or grey scale? Use different line types and/or markers

Plots: axis labelling

- Always indicate label for axis and units of measurement

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

149

Which data is your chart showing? Be explicit about the data on the axis and do not forget about units of measurement

Use a key for explaining which curves are displayed in the chart

Use a caption for your chart when it is included in a document, e.g.: Figure 1: <...>

Plots: axis scales

- Use coherent axis scales

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

150

Vertical axes should generally begin at the origin (zero). Visualizations in which the vertical axis does not begin at the origin can give a misleading picture of the meaning of the data.

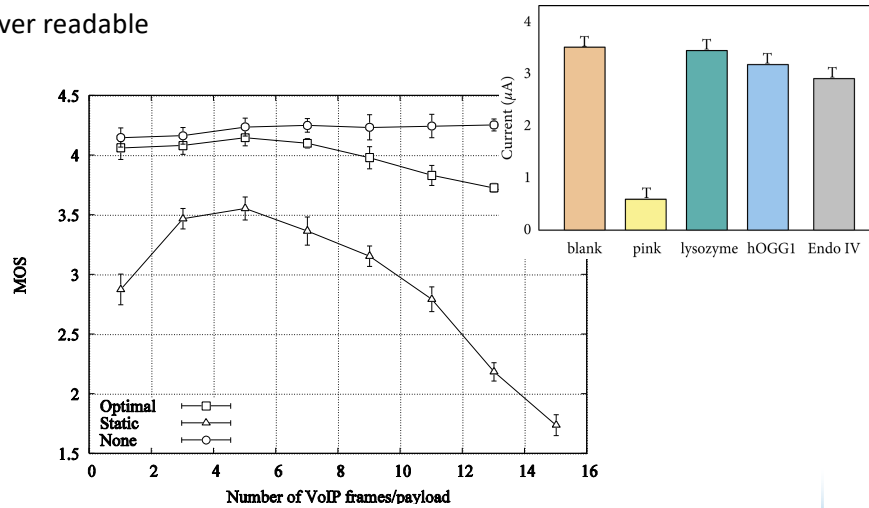
If your data describe a small effect you may need to employ a scale that makes the effect visible; however, you should make it very clear that your y-axis does not begin at the origin.

If you have multiple charts next to each other, try to use the same scales, otherwise the observer may be led to wrong conclusions

If your data are grouped into specific spans of time, the spans should be equal. For example, if you have time on the horizontal axis you need to use consistent scale divisions.

Confidence intervals

- Draw them whenever readable



Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

151

What confidence level should I use in my graphs? Good choices are 90%, 95% or 98% (the higher the confidence, the larger the interval).

If the intervals are too large, then measures may lose their meaningfulness. In this case, just **increase the number of independent replicas**, and see what happens.

Obviously enough, if you can rely on a **simulation automation software**, you can instruct it to perform as many replicas as required to have a small enough confidence interval (say, 10% of the mean), with a given maximum number (say, 30 – beyond that number you don't get any improvement).

There are cases when confidence intervals are **large**. It happens, for instance, when queueing systems are **close to saturation**, and the delays may vary greatly from one replica to another. In this case, there is little that you can do but acknowledge this fact.

Automate your methods!

- Use scripting
- Suggested software
 - Excel/Calc
 - Gnuplot
 - Pandas (Python)
 - MATLAB
 - ...

Simulation

Ing. Giovanni Nardini - University of Pisa - All rights reserved

152

More than one tool might be useful, possibly at different stages or for different purposes.

For instance:

- Excel is ready to use, especially when it comes to just plotting your data. However, it is not that good for repetitive actions
- Gnuplot is more complex to use, but has a scripting language and can be called from command line (aka, you can execute it from within other programs)
- Pandas is very powerful to manipulate data before plotting it, e.g. switching columns and rows of a table